

**EFFICIENT BARRIER OPTION PRICING IN JUMP DIFFUSION
FRAMEWORKS**

DANEEL PATEL

Dissertation submitted for the MSc in Mathematical Finance

Department of Mathematics
University of York

20 June 2025

Supervisor: Professor Maciej Capiński

CONTENTS

1. Introduction	3
2. Stochastic Calculus Preliminaries	4
3. Option Theory Preliminaries	10
4. Black Scholes Model	12
5. Running minimum/maximum of Wiener Process	14
6. Merton Jump Diffusion Model	17
7. Option Price Calculation	22
8. Algorithm for pricing call barrier option	33
8.1. Setup	33
8.2. Uniform Sampling algorithm	35
8.3. Taylor Series Approximation	37
9. Extension of Uniform Algorithm	40
10. Numerical Results	44
11. Conclusion	47
12. Appendix	49
References	60

1. INTRODUCTION

Accurate pricing of derivative contracts remains one of the most crucial tasks in modern financial markets. Derivatives not only underpin a wide variety of trading strategies but also play a central role in risk management, portfolio insurance, and capital allocation. Given the scale and speed of today's financial markets, the need for pricing models that are both theoretically sound and computationally efficient has become more pressing than ever. Mispricing even a small subset of derivatives can have significant consequences—leading to hedging errors, regulatory breaches, or even systemic risk, as illustrated by the Global Financial Crises of 2007-2008. Among the wide range of derivatives traded, barrier options occupy an exciting niche. These path-dependent instruments are highly sensitive to the trajectory of the underlying asset and are therefore used extensively in structured products and volatility trading strategies, however their valuation poses significant challenges.

While Black-Scholes-type models can be employed under certain assumptions, these often prove inadequate when markets are volatile or prone to sudden jumps. The mathematical complexity of barrier options, combined with the computational difficulty of simulating rare path features such as first-passage times, makes them a natural candidate for more sophisticated numerical methods. Moreover, the Black-Scholes framework assumes continuous and smooth price evolution, governed by Brownian motion. This assumption, while mathematically convenient, fails to capture the discontinuities observed in real-world asset prices. Empirical studies have shown that asset returns exhibit jumps—sudden large changes in price—often triggered by macroeconomic announcements, geopolitical tensions, or corporate events such as earnings surprises. These jumps introduce skewness and kurtosis into the return distribution, phenomena that continuous models cannot replicate. To address this limitation the Merton Jump Diffusion model extends the Black-Scholes framework by incorporating a Poisson-driven jump component. This allows for more realistic modelling of asset price dynamics, as it can capture both the usual day-to-day fluctuations and the sudden shifts that characterise financial markets. However, this added realism comes at a computational cost. The pricing of barrier options under Merton Jump Diffusion dynamics is considerably more complex: closed-form solutions are not available, and the naive application of Monte Carlo methods leads to inefficiencies, particularly when barriers are rarely hit.

In this dissertation, we seek to address this problem by studying and extending two advanced Monte Carlo algorithms designed specifically for pricing barrier options under the Merton Jump Diffusion model. These algorithms, originally proposed in [8], aim to reduce the time typically associated with simulating rare path-dependent events in the presence of jumps. However, the source provides only a sketch of their derivation and implementation. Our first major contribution is to derive these algorithms rigorously from first principles, grounding our analysis in stochastic calculus and probability theory. We begin by reviewing the necessary mathematical tools, including the Wiener process, its extremal properties, and Girsanov's theorem, all of which are instrumental in modelling barrier crossings and changing measures.

A key focus of our methodology is the use of the Brownian bridge to interpolate the asset path between discrete monitoring dates. This technique, when adapted to lognormal processes, enables more accurate detection of barrier crossings that might occur between time steps.

The first algorithm we explore is a uniform sampling method, where a point is sampled uniformly from an extended time interval; if this sampled point lies within the Brownian bridge segment, a crossing is assumed. The second algorithm employs a Taylor series expansion to approximate the integral expression for the probability of crossing. Although we are only able to derive the series partially, we validate its effectiveness through extensive numerical testing, using the trapezium rule to approximate the integral where needed.

We then extend the original approach by incorporating variance reduction techniques, particularly the use of a control variate tailored to the Merton Jump Diffusion setting. By carefully constructing a control variate with a known expected value, we are able to significantly reduce the estimator's variance without materially increasing computational complexity. This is especially beneficial when the barrier is far from the starting value of the asset, a setting in which standard Monte Carlo methods tend to perform poorly. Our simulation results reveal that these enhanced algorithms offer substantial improvements over basic Monte Carlo schemes. In many cases, we observe a variance reduction of nearly 50%, without increasing run-time significantly. This suggests that the techniques presented here could be directly applicable in industry settings, where computational resources are finite and timely pricing is essential. Through this work, we contribute practical, computationally efficient techniques for pricing barrier options in markets characterised by jumps, thereby enhancing the applicability of the Merton Jump Diffusion model in real-world financial settings. We implement all algorithms in C++ to allow for high-performance simulations and provide the code.

2. STOCHASTIC CALCULUS PRELIMINARIES

In this section, we'll cover the key concepts and definitions needed to develop the pricing algorithm for the Merton Jump-Diffusion model. We'll start from fundamental probability ideas and progress to more advanced tools like martingales and Ito processes, assuming the reader has some prior understanding of probability. To begin, we need a structure where randomness can live — this is what a probability space gives us.

Let $\mathcal{F}$ be a sigma field on a fixed set Ω , P be a probability measure we shall denote a probability space as $(\Omega, \mathcal{F}, P)$.

Definition 2.1. We will define an event as a subset from the sample space Ω within a measurable space $(\Omega, \mathcal{F})$, where $\mathcal{F}$ is the corresponding sigma field.

Now we will define the main object we'll work with: the stochastic process.

Definition 2.2. A stochastic process is a measurable mapping $X : \mathbb{T} \times \Omega \rightarrow \mathbb{R}$ with respect to the sigma field $\mathcal{B} \times \mathcal{F} = \sigma(\{B \times A : A \in \mathcal{F}, B \in \mathcal{B}_{\mathbb{T}}\})$, where $\mathbb{T}$ is an index set, $(\Omega, \mathcal{F}, P)$ is a probability space and $\mathcal{B}_{\mathbb{T}}$ is the Borel sigma field on $\mathbb{T}$.

As time evolves the information that is known will consequently change to represent this we will use a filtration.

Definition 2.3. A filtration is a family of sigma fields $(\mathcal{F}_t)_{t \in \mathbb{T}}$ where $\mathbb{T} = [0, T]$, such that for any $s, t \in \mathbb{T}$, if $s \leq t$, then

$$\mathcal{F}_s \subseteq \mathcal{F}_t.$$

Definition 2.4. Let X be a stochastic process and take $\mathcal{F}_{X(t)} = \{X(t)^{-1}(B) : B \in \mathcal{B}\}$. We will say that

$$\begin{aligned}\mathcal{F}_t^X &= \sigma(\cup_{s \leq t} \mathcal{F}_{X(s)}), \\ &= \cap \{G : G \text{ is a family of Sigma fields and } \mathcal{F}_{X(s)} \subset G \text{ for all } s \leq t\}.\end{aligned}$$

is a filtration generated by the process X .

Now that we've constructed a family—quite early on, I might add, though not the kind you'd have over for dinner—we need to understand what information we can derive from the sigma field for each random variable. This will help us introduce the concept of a martingale. But first, let's define the notion of a path.

Definition 2.5. Fix $\omega \in \Omega$. For stochastic process X , the map

$$t \rightarrow X(t, \omega) : \mathbb{T} \rightarrow \mathbb{R},$$

is called a trajectory or path of X corresponding with ω .

Definition 2.6. Let $(\Omega, \mathcal{F}, P)$ be a probability space and let $\mathcal{F}_t \subseteq \mathcal{F}$ be a sub-Sigma-field representing the information up to time t . A random variable X is said to be $\mathcal{F}_t$ - measurable if, for every Borel set $\mathcal{B} \subseteq \mathbb{R}$,

$$\{\omega \in \Omega : X(\omega) \in \mathcal{B}\} \in \mathcal{F}_t.$$

To introduce a martingale we must define what it means for a random variable to be adapted. (And no, not in the natural selection context — although randomness plays a role there too!)

Definition 2.7. Let $X(t)$ be a stochastic process, and $\mathcal{F}_t^X$ be a filtration generated by $X(t)$. The process $X(t)$ is said to be adapted if for every $t \in \mathbb{T}$ the random variable $M(t)$ is $\mathcal{F}_t$ - measurable.

Definition 2.8. Let $M(t)$ be a stochastic process and $(\mathcal{F}_t)_{t \in \mathbb{T}}$ a filtration to which $M(t)$ is adapted. For $s < t$ if:

$$\mathbb{E}(M(t) | \mathcal{F}_s) = M(s),$$

then $M(t)$ is called a martingale.

A neat property of a martingale is the conditional expectation which we will make use of when pricing options in the Black Scholes and Merton Jump Diffusion model.

Theorem 2.9. [4] *The expected value of a martingale at any time $t \in \mathbb{T}$ is equal to its initial value. Specifically for a martingale $M(t)$,*

$$\mathbb{E}(M(t)) = M(0).$$

Before introducing the Wiener process and presenting an Ito integral, it is useful to first establish some foundational tools that will facilitate the upcoming derivations. In particular, we review the concepts of joint and conditional density functions as well as the Laplace transform and its key properties. These tools will be instrumental in understanding the probabilistic structure of stochastic processes and in simplifying certain calculations that arise in stochastic calculus.

Definition 2.10. Let X and Y be continuous random variables defined on probability space (Ω, F, P) . The joint density function of X and Y denoted as

$$f_{X,Y}(x, y),$$

is a map

$$f_{X,Y} : \mathbb{R}^2 \rightarrow [0, \infty).$$

For any Borel sets $A, B \subseteq \mathbb{R}$

$$P(X \in A, Y \in B) = \int_A \int_B f_{X,Y}(x, y) dy dx.$$

Definition 2.11. Let X and Y be continuous random variables defined on the probability space (Ω, F, P) , with joint density function $f_{X,Y}(x, y)$, and assume that the marginal density $f_Y(y) > 0$ for all $y \in \mathbb{R}$. The conditional density function of X given by $Y = y$, denoted

$$f_{X|Y}(x | y),$$

is defined by:

$$f_{X|Y}(x | y) = \frac{f_{X,Y}(x, y)}{f_Y(y)}.$$

This function satisfies:

$$P(X \in A | Y = y) = \int_A f_{X|Y}(x | y) dx \text{ for any Borel set } A \subseteq \mathbb{R}.$$

We employ the Laplace transform in conjunction with Taylor's theorem to approximate a key integral that features prominently in one of the algorithms. Some of the results below are taken directly from the textbook source, while others are derived independently. For the latter, we restate the relevant lemmas in the appendix and provide full proofs.

Definition 2.12. Let $\mathcal{L}[x(t)]$ denote the Laplace transform of a function $x(t)$ defined is by the integral:

$$\mathcal{L}[x] = \int_0^\infty x(t) \exp(-st) dt.$$

Theorem 2.13. [5] *The Laplace transform of the function $x(t) = t^{-\frac{3}{2}} \exp(\frac{-b}{4t})$ is given by:*

$$(2.1) \quad \mathcal{L} \left[t^{-\frac{3}{2}} \exp(\frac{-b}{4t}) \right] = 2\sqrt{\pi} b^{-\frac{1}{2}} \exp(\sqrt{-bs}).$$

Theorem 2.14. [5] *The Laplace transform of the function $x(t) = t^{-\frac{1}{2}} \exp(\frac{-b}{4t})$ is given by:*

$$(2.2) \quad \mathcal{L} \left[t^{-\frac{1}{2}} \exp(\frac{-b}{4t}) \right] = \sqrt{\pi} s^{-\frac{1}{2}} \exp(\sqrt{-bs}).$$

Lemma 2.15. [5] *The Laplace transform of the function $x(t) = at^{-\frac{3}{2}} \exp(\frac{-a^2}{4t})$ is given by:*

$$(2.3) \quad \mathcal{L} \left[at^{-\frac{3}{2}} \exp(\frac{-a^2}{4t}) \right] = 2\sqrt{\pi} \exp(-a\sqrt{s}).$$

Lemma 2.16. *The Laplace transform of the function $x(t) = t^{-\frac{1}{2}} \exp\left(\frac{-a^2}{4t}\right)$ is given by:*

$$(2.4) \quad \mathcal{L}\left[t^{-\frac{1}{2}} \exp\left(\frac{-a^2}{4t}\right)\right] = \frac{\sqrt{\pi} \exp(-|a|\sqrt{s})}{\sqrt{s}}.$$

Definition 2.17. Let $f(t)$ and $g(t)$ be two functions define on $[0, \infty)$. Their convolution denoted by $(f * g)(t)$ is defined as:

$$(f * g)(t) = \int_0^t f(\tau)g(\tau - t)d\tau.$$

Theorem 2.18. [5] *(Convolution Property of Laplace Transform) Let $f(t)$ and $g(t)$ be two functions such that their Laplace transforms $\mathcal{L}[f(t)] = F(s)$ and $\mathcal{L}[g(t)] = G(s)$ exist for $s \in \mathbb{C}$. Then we can express the Laplace transform of the their convolution as the produce of their Laplace transforms:*

$$\mathcal{L}[(f * g)(t)] = F(s)G(s),$$

where the convolution integral converges for $t \geq 0$.

Theorem 2.19. [5] *(Time Shift Property of Laplace Transform) Let $f(t)$ be a function such that its Laplace transforms $\mathcal{L}[f(t)] = F(s)$ exist for $s \in \mathbb{C}$. Then for all $a \in \mathbb{R}$ the Laplace transform $f(t)$ multiplied by $\exp(-at)$ is given by the following:*

$$\mathcal{L}[f(t) \times \exp(-at)] = F(s + a).$$

For all of the above, it is assumed that $s \in \mathbb{C}$ and the integral defining the Laplace transform converges.

We now introduce the main star of randomness — the Wiener Process, also known as Brownian motion. This process forms the backbone of continuous-time stochastic calculus, and will appear frequently in everything from martingales to stochastic differential equations.

Definition 2.20. A Wiener Process (Brownian motion) is a mapping $W : [0, \infty) \times \Omega \rightarrow \mathbb{R}$ defined on a probability space $(\Omega, \mathcal{F}, P)$, measurable with respect to the product sigma field $\mathcal{B}[0, \infty) \times \mathcal{F}$ where $\mathcal{B}[0, \infty)$ is the sigma field of Borel subsets of $[0, \infty)$ such that:

- (1) $W(0) = 0$.
- (2) The increments have a normal distribution $W(t) - W(s) \sim N(0, t - s)$ where $0 \leq s < t < \infty$.
- (3) All increments are independent.
- (4) Almost all paths are continuous.

Before diving deeper, it's worth highlighting a key property of Brownian motion: it satisfies the Markov property, meaning that its future behaviour depends only on its present value, not the full history.

Theorem 2.21. [10] *[Markov Property] Let $W(t)$ be a Wiener process defined on a probability space $(\Omega, \mathcal{F}, P)$ with its natural filtration $\{\mathcal{F}_t\}_{t \geq 0}$ where $\mathcal{F}_t = \sigma(W(s) : 0 \leq s \leq t)$. Then for any $0 \leq s < t$ and any Borel-measurable function $f : \mathbb{R} \rightarrow \mathbb{R}$, such that the expectation exists and is finite, we have:*

$$\mathbb{E}[f(W(t)) \mid \mathcal{F}_s] = \mathbb{E}[f(W(t)) \mid W(s)].$$

Equivalently, the conditional distribution of $W(t)$ given the past $W(u) : u < s$ depends only on the current state $W(s)$. That is

$$P(W(t) \in A \mid \mathcal{F}_s) = P(W(t) \in A \mid W(s)).$$

This expresses the Markov property of a Wiener Process.

This is the formal way of saying “the future is conditionally independent of the past, given the present.” This makes Brownian motion memoryless, like a continuous-time version of a fair coin toss. To study the path-wise behaviour of Brownian motion, we introduce the quadratic variation, which measures how “rough” the process is.

Definition 2.22. Let $0 = t_0 < t_1 < t_2 \dots < t_n = T$. We will use the notation $\pi = \{t_1, \dots, t_n\}$ and define mesh size of the partition as $|\pi(T)| = \max_{i=1 \dots n} |t_i - t_{i-1}|$. We will refer to $\pi(T)$ as a partition of the interval $[0, T]$.

For a stochastic process X define

$$V_{\pi(T)}^2(X) = \sum_{i=0}^{n-1} (X(t_{i+1}, \omega) - X(t_i, \omega))^2.$$

If for every sequence of partitions $\pi_n(t)$ satisfying $\lim_{n \rightarrow \infty} |\pi_n(t)| = 0$ the

$$[X, X](t) = \lim_{n \rightarrow \infty} V_{\pi_n(t)}^2(X),$$

is well defined P a.s. and independent from the choice of the sequence $\pi_n(t)$, then we call $[X, X](t)$ the quadratic variation of X .

Theorem 2.23. [4] *The Quadratic Variation of the Wiener Process $W(t)$ is given by:*

$$[W, W](t) = t.$$

Theorem 2.24. [4] *If $W(t)$ is a Wiener process then for all $0 \leq s < t < u$, the random variable $W(u) - W(t)$ is independent of $\mathcal{F}_s^W$.*

Theorem 2.25. [4] *The following processes are martingales with respect to $\mathcal{F}_t^W$ where $W(t)$ is a Wiener Process:*

- (1) $W(t)$.
- (2) $W^2(t) - t$.
- (3) $\exp\{\sigma W(t) - \frac{1}{2}\sigma^2 t\}$.

These are classic examples that showcase different martingale structures. The exponential martingale, in particular, plays a central role in Girsanov’s Theorem and change of measure, which we will define later. Before we can define stochastic integrals, we need to clarify the types of processes we can integrate.

Definition 2.26. The square integrable space of adapted processes is defined as:

$$\mathcal{M}^2 = \left\{ X : [0, T] \times \Omega \rightarrow \mathbb{R} : X \text{ is adapted to } \mathcal{F}_t^W \text{ and } E \left(\int_0^T X^2(t) dt \right) \leq \infty \right\}.$$

Definition 2.27. Let $X \in \mathcal{M}^2$. We say that X is simple and belongs to $\mathcal{S}^2$ if there exists a partition $0 = t_0 < t_1 < t_2 < \dots < t_k = T$ of $[0, T]$ and a sequence of $\mathcal{F}_{t_i}$ -measurable, square integrable random variables λ_i , for $i = 0, \dots, k$, such that

$$X(t, \omega) = \lambda_0(\omega)1_{[t_0, t_1]}(t) + \sum_{i=1}^{k-1} \lambda_i(\omega)1_{(t_i, t_{i+1}]}(t).$$

Here $1_A(t)$ denotes the indicator function of set A .

Theorem 2.28. [4] *For every continuous process $X \in \mathcal{M}^2$ there exists a sequence of simple processes X_n for $n \geq 1$ such that:*

$$\lim_{n \rightarrow \infty} E \left(\int_0^T (X_n(t) - X(t))^2 dt \right) = 0.$$

Theorem 2.28 ensures that even though real-world stochastic processes may be complex, we can approximate them arbitrarily well using building blocks — simple, piecewise-constant adapted processes. This is the foundation for defining the Ito integral.

Definition 2.29. The Ito Integral of process $X \in \mathcal{S}^2$ is denoted as:

$$I(X) = \int_0^T X(t) dW(t) = \sum_{i=0}^{k-1} \lambda_i (W(t_{i+1}) - W(t_i)).$$

Definition 2.30. The Ito Integral of process $X \in \mathcal{M}^2$ is denoted as:

$$I(X) = \int_0^T X(t) dW(t),$$

and defined as follows:

For every sequence $X_n \in \mathcal{S}^2$ such that

$$E \left(\int_0^T (X(t) - X_n(t))^2 dt \right) < \infty \text{ as } n \rightarrow \infty,$$

the corresponding sequence of stochastic integrals $I(X_n)$ converges in $L^2(\Omega)$ then $I(X)$ is defined as

$$I(X) = \lim_{n \rightarrow \infty} I(X_n).$$

Theorem 2.31. [3] *The Ito integral $I(X)$ exists and is well defined for every $X \in \mathcal{M}^2$.*

For $t \leq T$ we will use the notation

$$\int_0^t X(s) dW(s) = I(1_{[0,t]} X),$$

where $1_{[0,t]}$ is the indicator function over the interval $[0, t]$.

Theorem 2.32. [4] *For $X \in \mathcal{M}^2$ we have following properties:*

- (1) *We have isometry $\mathbb{E} \left(\left[\int_0^T X(t) dW(t) \right]^2 \right) = \mathbb{E} \left(\int_0^T X^2(t) dt \right)$.*
- (2) $\mathbb{E} \left(\int_0^T X(t) dW(t) \right) = 0$.
- (3) *If Y is $\mathcal{F}_a^W$ -measurable and bounded, then $\int_a^b Y X(s) dW(s) = Y \int_a^b X(s) dW(s)$.*
- (4) *The Quadratic Variation of $Y(t) = \int_0^t X(s) dW(s)$ is $[Y, Y](t) = \int_0^t X^2(s) ds$.*

We now define a broad class of processes known as Ito processes, which form the foundation of stochastic calculus:

Definition 2.33. An Ito process $X(t)$ has the form:

$$X(t) = X(0) + \int_0^t a(s)ds + \int_0^t b(s)dW(s) \text{ for } t \in [0, T],$$

$$dX(t) = a(t)dt + b(t)dW(t),$$

where $a(s)$ is the drift, satisfying

$$E \left(\int_0^t |a(s)|ds \right) < \infty,$$

and $b(s) \in \mathcal{M}^2$ is the diffusion coefficient.

Theorem 2.34. [4] *If X is an Ito process then:*

- (1) *Almost all paths of X are continuous.*
- (2) *If $a = 0$, then X is a martingale.*
- (3) *If $\int_0^t a(s)ds < \infty$ then $\mathbb{E}(X(t)) = X(0) + \int_0^t \mathbb{E}[a(s)]ds$.*
- (4) $[X, X](t) = \int_0^t b^2(s)ds$.

Finally, we introduce the powerful Ito Formula, which allows us to compute the dynamics of functions of Ito processes.

Theorem 2.35. [4] *Take $dX(t) = a(t)dt + b(t)dW(t)$ if there exists $F : \mathbb{R} \rightarrow \mathbb{R}$ such that the first and second differentials exist and are continuous and $\int_0^T a(t) < \infty$ and $b(t) \in \mathcal{M}^2$ then $F(t, X(t))$ is an Ito process and the following formula holds:*

$$\begin{aligned} dF(t, X(t)) &= F'(t, X(t))dX(t) + \frac{1}{2}F''(t, X(t))[X, X](t), \\ &= (F_t(t, X(t)) + F_x(t, X(t))a(t) + F_x(t, X(t))b(t))dW(t) \\ &\quad + \left(\frac{1}{2}F_{xx}(t, X(t))b^2(t) \right) dt. \end{aligned}$$

3. OPTION THEORY PRELIMINARIES

The concept of options dates back to ancient times, with early forms of option-like contracts reportedly used by Greek philosophers such as Thales to secure the rights to olive presses. In modern financial markets, options have evolved into a central class of derivative instruments used for hedging, speculation, and risk management. In this section, we introduce the formal definition of barrier options and describe their key features. We also present the mathematical framework used to characterise their payoffs, which forms the foundation for the pricing techniques discussed later in the dissertation.

Definition 3.1. An option is a financial derivative that gives the holder the right, but not the obligation, to buy or sell an underlying asset at a predetermined price (called the strike price) on or before a specified date (called the expiry or maturity date). There are two primary types of options:

- (1) A call option gives the holder the right to buy the underlying asset at the strike price.
- (2) A put option gives the holder the right to sell the underlying asset at the strike price.

Traders often talk about whether an option is “in the money” or “out of the money”—which simply refers to whether exercising the option would currently be profitable. This terminology helps to quickly assess the option’s current value relative to the strike price.

Definition 3.2. Let $S(t)$ denote the current price of the underlying asset, and K represent the strike price of an option:

- A Call option is said to be in the money if $S(t) > K$ and out of the money if $S(t) < K$.
- A Put option is said to be in the money if $S(t) < K$ and out of the money if $S(t) > K$.

Definition 3.3. A path-dependent derivative is a type of financial instrument whose payoff depends not only on the final value of the underlying asset at maturity but also on the path taken by the asset price over the life of the contract. That the derivative’s value is influenced by the entire trajectory of the Stock Price, $S(t)_{t \in [0, T]}$, of the underlying asset price process.

Now onto the type of option we will be pricing in this dissertation, a Barrier option.

Definition 3.4. A Barrier option is a path-dependent financial derivative whose payoff depends on whether the price of the underlying asset $S(t)$ crosses a predefined barrier level H during the option’s lifetime $[0, T]$. These options are categorised into two primary types:

- (1) Knock-In: The option is in the money only if $S(t)$ crosses H before expiry.
- (2) Knock-Out: The option is out of the money if $S(t)$ crosses H before expiry.

The final payoff is determined by both the barrier-triggering event and the terminal price $S(T)$. If the option is rendered inactive (e.g., a knock-out option is triggered or a knock-in option is never activated), a predetermined rebate R may be paid to the holder at expiry T ,

Barrier options are further classified based on the position of the barrier relative to the initial asset price $S(0)$:

- A down-and-out option is a knock-out option where the barrier H is set below the initial asset price. The option becomes void if the underlying asset price $S(t)$ falls to or below the barrier H at any time during $[0, T]$.
- A up-and-out option option is a knock-out option where the barrier H is set above the initial asset price. The option becomes void if the underlying asset price $S(t)$ rises to or above the barrier H at any time during $[0, T]$.
- A down-and-in option is a knock-in option where the barrier H is set below the initial asset price. The option becomes void if the underlying asset price $S(t)$ falls to or below the barrier H at any time during $[0, T]$.
- A up-and-in option is a knock-in option where the barrier H is set below the initial asset price. The option becomes void if the underlying asset price $S(t)$ rises to or above the barrier H at any time during $[0, T]$.

Below we have an example of a the payoff structure of a down and out knock-out call option, that pays rebate R when there is a barrier crossing, it is worth noting that not all barrier options pay a rebate, however the down and out knockout call option that we will be providing algorithms for pays a rebate.

Example 3.5. Let $\tau = \inf \{t \in [0, T] : S(t) \leq H\}$, the payoff for a down and out knock-out call option

$$\text{Payoff} = \begin{cases} R & \text{if } \tau \neq \emptyset, \\ S(T) - K & \text{if } \tau = \emptyset \text{ and } S(T) > K. \end{cases}$$

Hitting the barrier means that

$$S(t) = H,$$

which is equivalent to

$$x(t) = \ln S(t) = \ln H.$$

Crossing a barrier is denoted as:

$$\min_{t \in [0, T]} x(t) < \ln H < \max_{t \in [0, T]} x(t).$$

4. BLACK SCHOLES MODEL

The Black-Scholes model, developed in the early 1970s by Fischer Black, Myron Scholes, and later extended by Robert Merton, marked a turning point in the field of financial mathematics. Prior to their work, the pricing of options and other derivatives relied heavily on heuristics, market conventions, or simplistic assumptions that lacked a solid mathematical foundation. In 1973, Black and Scholes published their groundbreaking paper "The Pricing of Options and Corporate Liabilities", in which they introduced a partial differential equation (now known as the Black-Scholes PDE) that could be used to determine the fair price of European-style options under certain assumptions. Merton independently contributed by generalising and rigorously proving the solution using stochastic calculus, and also popularised the model in academic and practitioner circles. Not only did the model revolutionise derivative pricing, but it also laid the foundation for the modern field of quantitative finance. The impact was so profound that Scholes and Merton were awarded the Nobel Prize in Economic Sciences in 1997.

We first present the model used to describe the evolution of the stock price over time:

Definition 4.1. The value of a stock at time t as $S(t)$ under the dynamics:

$$(4.1) \quad dS(t) = \mu S(t)dt + \sigma S(t)dW(t),$$

where we assume $S(t) \in \mathcal{M}^2$ and denote μ as the average return of the stock and σ denotes the standard deviation of the stock.

This is the famous geometric Brownian motion. In simple terms: the stock is trying to grow at rate μ , but the Wiener process is constantly messing with its plans.

We can now introduce the notion of discounting.

Definition 4.2. The discounted stock price is defined as

$$\tilde{S}(t) = S(t) \exp(-rt),$$

where $S(t)$ is the stock price at time t and r is the risk free rate.

Theorem 4.3. [3] *The discounted stock price at time t , $\tilde{S}(t)$, has the dynamics*

$$(4.2) \quad d\tilde{S}(t) = (\mu - r)dt + \sigma dW(t).$$

Theorem 4.4. [3] *The filtration $\mathcal{F}_t^S$ coincides with $\mathcal{F}_t^W$.*

The stock isn't hiding any secrets — all its randomness comes from the Brownian motion. No insider information sneaking in through the back door. The solution to the dynamics Equation (4.1) and Equation (4.2) is as follows.

Theorem 4.5. [4] *Let $S(t)$ be the value of the stock as time t with dynamics (4.1) then there exist the unique solutions:*

$$S(t) = S(0) \exp \left(\mu t - \frac{1}{2} \sigma^2 t + \sigma W(t) \right),$$

$$\tilde{S}(t) = S(0) \exp \left((\mu - r)t - \frac{1}{2} \sigma^2 t + \sigma W(t) \right),$$

where $\tilde{S}(t)$ is the value of the discounted stock.

This is one of the rare moments in stochastic calculus where you get a neat closed-form solution. Cherish it — it doesn't happen often.

Before diving into Girsanov's Theorem, it's worth asking a simple but profound question: how do we price risky assets in a world where everyone dislikes risk?

In reality, investors demand a premium for bearing uncertainty — but in the risk-neutral world, we pretend they don't. This world isn't just a fantasy; it's a mathematical tool that lets us price derivatives by taking expected values under a new probability measure, where all assets grow at the risk-free rate.

But how can we legally switch between these two worlds? That's where Girsanov's Theorem comes in. It's the passport that lets us change from the real-world probability measure P to the risk-neutral measure Q , while keeping Brownian motion — just with a different drift. Think of it as re-weighting probabilities so that risky assets behave as if they were risk-free — at least on paper.

Now that we've got the motivation, let's formally introduce the risk-neutral world and Girsanov's Theorem.

Definition 4.6. Probability measures P and Q both defined on $\mathcal{F}$ are equivalent when

$$P(A) = 0 \quad \Longleftrightarrow \quad Q(A) = 0,$$

for any $A \in \mathcal{F}$.

Definition 4.7. We say that Q is a risk neutral measure if it is equivalent to P and $\tilde{S}(t)$ is a martingale under the measure Q .

Theorem 4.8. [4] (*Girsanov's Theorem*)

Assume $W_Q = \int_0^t a(s)ds + W(t)$. If

$$M(t) = \exp \left(- \int_0^t a(s)dW(s) - \frac{1}{2} \int_0^t a^2(s)ds \right),$$

is a martingale and $Q(A) = \int_A M(T)dP$ then W_Q is a Wiener process under the measure Q .

Theorem 4.9. If $a(s) = \frac{\mu-r}{\sigma}$ then using Theorem 4.8 we get the dynamics equation for a stock:

$$dS(t) = rS(t)dt + \sigma S(t)dW_Q(t),$$

where $W_Q(t) = \left(\frac{\mu-r}{\sigma}\right)t + W(t)$ and is a Wiener process under measure Q .

5. RUNNING MINIMUM/MAXIMUM OF WIENER PROCESS

To compute the first-crossing density of a stock $S(t)$ for each Brownian bridge, which will be defined in a later section, it is useful to consider the running maximum of the stock. However, for ease of calculation, it suffices to work with the Wiener process associated with the stock and, consequently, the running maximum of this Wiener process. This allows us to apply Girsanov's Theorem 4.8 to transform the process accordingly.

Definition 5.1. Let $M(t)$ denote the running maximum of a Wiener Process $W(t)$ defined by:

$$M_t = \sup_{0 \leq s \leq t} W(s).$$

Definition 5.2. Let $W(t)$ be a standard Wiener Process. We define the first passage (crossing) time of the level $b \in \mathbb{R}$ as the stopping time:

$$\tau_b = \inf \{t \geq 0 : W(t) = b\}.$$

We can now introduce the reflected Wiener process which properties are very interesting and will help us compute the densities needed for the pricing algorithm.

Definition 5.3. For $t \geq 0$ the reflected process $\tilde{W}(t)$ is denoted as:

$$\tilde{W}(t) = \begin{cases} W(t) & \text{if } t \leq \tau_b, \\ 2W(\tau_b) - W(t) & \text{otherwise.} \end{cases}$$

Theorem 5.4. [3] Take $W(t)$ as standard Wiener Process and $M(t)$ to be the running maximum of $W(t)$ from $t \in [0, T]$. Let τ_b be a stopping time and $\tilde{W}(t)$ be a reflected process, then the following equivalence holds:

$$P(M(T) \geq b, W(T) \leq a) \Leftrightarrow P(\tilde{W}(t) \geq 2b - a).$$

Proof. For the forward case, assume that $M(T) \geq b, W(T) < a$, as $M(T) \geq b$.

Since $M(T) \geq b$, then there exists $\tau_b < T$ such that $W(\tau_b) = b$. By the definition of the reflected process for $t > \tau_b$, we have

$$\tilde{W}(t) = 2b - W(t).$$

Since $W(T) < a$, it follows that

$$-W(T) \geq -a \Leftrightarrow 2b - W(T) \geq 2b - a,$$

thus,

$$\tilde{W}(t) > 2b - a.$$

For the backward case, assume that $\tilde{W}(t) > 2b - a$. If $\tau_b > T$, then by definition of the reflective process we have that $\tilde{W}(t) = W(t)$, which implies that $W(T) \geq 2b - a$. However, this contradicts the fact that $W(t)$ must reach b before T , due to the continuity of the Wiener process. Therefore $\tau_b < T$ ensuring that $M(t) \geq b$. Since $\tilde{W}(T) = 2b - W(T) \geq 2b - a$, we rearrange and obtain

$$W(T) \leq a.$$

Hence we can conclude $M(T) \geq b$ and $W(T) \leq a$. □

Finally we can compute some densities!

Lemma 5.5. [9] *The probability that the running maximum $M(T)$ of a Wiener Process $W(t)$ exceeds a level b , given that $W(0) = 0$ and $W(T) = a$, is:*

$$P(M(T) \geq b \mid W(0) = 0, W(T) = a) = \exp\left(\frac{-(2b - a)^2 + a^2}{2t}\right).$$

Where $M(T)$ is the running maximum of the process over the interval $[0, T]$, and b is the threshold level for the maximum.

Proof. To derive the desired density, we begin by applying Bayes' Theorem to the conditional probability

$$P(M(T) \geq b \mid W(0) = 0, W(T) = a).$$

Since the Wiener process $W(t)$ is almost surely continuous, the event $W(T) = a$ has probability zero. Therefore, when applying Bayes' Theorem, we consider instead the probability of $W(T) \in [a, a + \delta]$ where δ is an arbitrarily small positive increment. This allows us to work with a density rather than a point probability.

$$P(M(T) \geq b \mid W(0) = 0, W(T) = a) = \frac{P(M(T) \geq b, W(T) \in [a, a + \delta])}{P(W(T) \in [a, a + \delta])}.$$

Using Theorem 5.4 for Brownian motion, we have:

$$\begin{aligned} P(M(T) \geq b, W(T) \in [a, a + \delta]) &= P(W(T) \in [2b - a + \delta]), \\ &= \frac{1}{\sqrt{2\pi T}} \exp\left(\frac{-(2b - a)^2}{2T}\right). \end{aligned}$$

From independent and normal distribution of Wiener increments 2 we get:

$$P(W(T) \in [a, a + \delta]) = \frac{1}{\sqrt{2\pi T}} \exp\left(\frac{-a^2}{2}\right).$$

Substituting the above expressions in the conditional probability formula:

$$\begin{aligned} P(M(T) \geq b \mid W(0) = 0, W(T) = a) &= \frac{\frac{1}{\sqrt{2\pi T}} \exp\left(\frac{-(2b - a)^2}{2T}\right)}{\frac{1}{\sqrt{2\pi T}} \exp\left(\frac{-a^2}{2T}\right)}, \\ &= \exp\left(\frac{a^2 - (2b - a)^2}{2T}\right). \end{aligned}$$

□

Having established the properties of the maximum, the corresponding properties of the minimum follow in a straightforward manner.

Definition 5.6. We define the running minimum of a Wiener Process $W(t)$ defined over $[0, T]$ as $m(t)$ and this is denoted as :

$$m(t) = \inf_{t \in [0, T]} W(t).$$

Corollary 5.7. *Let $M(t)$ denote the running maximum and $m(t)$ denote the running minimum of Wiener process $W(t)$ then*

$$P(M(t) < b) = P(-m(t) < b).$$

Proof. Let $M(t)$ denote the running maximum and $m(t)$ denote the running minimum of Wiener process $W(t)$.

The running maximum $M(t)$ is written as

$$M(t) = \sup_{s \in [0, t]} W(s).$$

Similarly we define the running minimum $m(t)$ as:

$$m(t) = \inf_{s \in [0, t]} W(s).$$

Consider the process $-W(t)$. Since $-W(t)$ is also a Wiener process and as $W(t)$ is symmetric in distribution we can write:

$$W(t) \stackrel{d}{=} -W(t),$$

where $\stackrel{d}{=}$ denotes equal in distribution. For $-W(t)$ the supremum of the process can be expressed in terms of the infimum of $W(t)$:

$$\sup_{s \in [0, t]} -W(s) = - \inf_{s \in [0, t]} W(s),$$

$$M(t) \stackrel{d}{=} -m(t).$$

Using the above relationship we can rewrite $P(M(t) < b)$ as:

$$P(M(t) < b) = P(-m(t) < b).$$

□

Lemma 5.8. [9] *The probability density function for the first passage time τ_a of a Wiener Process $W(t)$ to reach a specified level $a > 0$ is given by:*

$$P(\tau_a \in [t, T]) = \int_t^T \frac{a}{\sqrt{2\pi(T-t)^3}} \exp\left(-\frac{a^2}{2(T-t)}\right) dt,$$

where τ_a is the first time the Wiener process $W(t)$ hits point a and $a > 0$ is the target level to be reached by the process.

Proof. From the Theorem 5.4 we know that

$$P(\tau_a \leq a) = 2P(W(t) \geq a).$$

We want to calculate the density so differentiating the above function should suffice for the proof.

Let's denote

$$P(W(t) \leq b) = \Phi\left(\frac{b}{\sqrt{t}}\right),$$

such that $\Phi(x)$ is the standard normal cumulative density function.

To get the density we shall differentiate:

$$\begin{aligned} \frac{\delta}{\delta t} (2P(W(t) \geq a)) &= \frac{\delta}{\delta t} 2 \left(1 - \Phi\left(\frac{a}{\sqrt{t}}\right)\right) \\ &= -\Phi\left(\frac{t}{a}\right) \times \left(-\frac{a}{2t^{\frac{3}{2}}}\right). \end{aligned}$$

The above is derived using the chain rule and we can expand out getting the desired density as:

$$\frac{a}{\sqrt{2\pi(T-t)^3}} \exp\left(-\frac{a^2}{2(T-t)}\right).$$

□

This will serve us well in getting the density of the event of a stock crossing a barrier in a down and out option.

6. MERTON JUMP DIFFUSION MODEL

To extend the dynamics of the Black-Scholes model to include discontinuous jumps, we need to introduce Poisson processes. Below, we will construct a Poisson Process $N(t)$, which counts the number of jumps that occur up to time t , assuming each jump has a size of 1. Hang in there — we're nearly at the exciting part: the algorithms!

Lets start with exponential distributions, they are like unpredictable alarm clocks — they don't tick regularly, but suddenly ring to signal that something random has happened. In our case: a jump!

Definition 6.1. We will say that a random variable r has exponential distribution if

$$P(r \leq x) = \begin{cases} 1 - e^{-\lambda x} & \text{for } x \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

If we want to emphasise the dependence on λ then we will say that r is λ -exponentially distributed.

Now that we know how long we wait between jumps, the Poisson process counts how many have happened — think of it as a jump counter clicking away through time.

Definition 6.2. Let $r_1, r_2, \dots$ be a sequence of independent λ -exponentially distributed random variables. Let

$$s(n) = r_1 + r_2 + \dots + r_n.$$

We define the Poisson process $N(t)$ as

$$N(t) = k \iff s(k) \leq t < s(k+1).$$

If we want to emphasise the dependence of λ then we will say that $N(t)$ has intensity λ .

Definition 6.3. We say that Poisson Process $N(t)$ has intensity λ when:

$$\mathbb{E}(N(t)) = \lambda t.$$

Definition 6.4. A Compounded Poisson Process $J(t)$ is a counting process that has variable jump sizes of $Y_i(t)$:

$$J(t) = \sum_{i=0}^{N(t)} Y_i(t) \text{ where } N(t) \text{ is a Poisson Process.}$$

We can now calculate the mean and variance of this process.

Theorem 6.5. [6] Take $J(t)$ to be a Compounded Poisson Process, where $N(t)$ has λ intensity and $Y_i(t)$ are independently identically distributed with mean μ_Y and variance σ_Y^2 then the mean and variance of $J(t)$ is given by:

$$\mathbb{E}(J(t)) = \lambda t \mu_Y.$$

$$\text{Var}(J(t)) = \lambda t (\sigma_Y^2 + \mu_Y^2).$$

Assuming $N(t)$ is independent of $Y_i(t)$ for all $i \in \mathbb{N}$.

In the Merton Jump Diffusion Model there are discontinuous jumps that need to be modelled, which will be achieved using a Poisson process from Definition 6.4. Unlike regular Poisson processes, which assume every jump is the same size, this guy lets each jump pack a different punch — just like real markets.

We can now introduce the Ito-Lévy (jump) process using the above definitions.

Definition 6.6. Process $X(t)$ is an Ito-Levy (jump) process if it can be written as

$$(6.1) \quad X(t) = X(0) + I(t) + R(t) + J(t).$$

Where

- $X(0)$ is the deterministic initial condition.
- $I(t)$ is a stochastic integral:
 - $I(t) = \int_0^t \Gamma(s) dW(s)$ such that $\Gamma(s) \in \mathcal{M}^2$ and adapted.
- $R(t)$ is a Riemann integral:
 - $R(t) = \int_0^t \Theta(s) ds$ such that $\int_0^t \Theta(s) ds < \infty$ and $\Theta(s)$ is adapted.
- $J(t)$ is a Compounded Poisson process.

This is the all-in-one model: drift + diffusion + jumps — the triple threat of stochastic modelling. To link an Ito process links with the Ito-Levy process we present the following.

Definition 6.7. Take $X(t)$ of from equation (6.1) we define the continuous part of the Ito-Levy process as $X^c(t)$ such that:

$$(6.2) \quad X^c(t) = X(0) + I(t) + R(t),$$

where $X(0)$ is the deterministic initial condition, $I(t)$ is a stochastic integral, $R(t)$ is a Riemann integral and $X^c(t)$ is an Ito process.

Definition 6.8. Let $X(t)$ be a jump process of form 6.1 and let $\phi(s)$ be an adapted process. The stochastic integral of ϕ with respect to X is defined as:

$$\int_0^t \phi(s) dX(t) = \int_0^t \phi(s) \Gamma(s) dW(s) + \int_0^t \phi(s) \Theta(s) ds + \sum_{0 < s \leq t} \phi(s) \Delta J(s),$$

where $\Delta J(s)$ is the size of the discontinuous jump at s .

To include jumps we must be able to distinguish between the discontinuous jump points.

Definition 6.9. Take $S(t)$ to be a stock process we shall write $S(t-)$ as the left-hand limit of S at time t , formally defined as:

$$S(t-) = \lim_{u \rightarrow t-} S(u).$$

This is the value of the process just before any jump at time t .

Similarly $S(t+)$ denotes the right- hand limit of S at time t , formally defined as:

$$S(t+) = \lim_{u \rightarrow t^+} S(u).$$

This is the value of the process just before any jump at time t .

Now this is the big one — the Ito formula with jumps! It's like the original Ito formula got a PhD in market realism, now accounting for both smooth paths and surprise leaps!

Theorem 6.10. [10] *Let $X(t)$ be a jump diffusion process. Assume there exists a function $F : \mathbb{R} \rightarrow \mathbb{R}$ such that $F(x) \in C^1, C^2$ [1st and 2nd differentials exist and are continuous]. Additionally suppose the integral conditions $\int_0^T a(t) < \infty$ holds and $b(t) \in \mathcal{M}^2$. Under these conditions $F(X, t)$ is an Ito-Levy process and the following formula applies:*

$$\begin{aligned} dF(X(t), t) &= F'(X(t), t) dX^c(t) + \frac{1}{2} F''(X(t), t) [X^c(t), X^c(t)] \\ &+ \sum_{0 \leq s \leq t} F(X(s)) - F(X(s-)), \end{aligned}$$

such that $X(s-)$ denotes the left-hand limit of X at time s .

Now that we have extended Ito's Lemma to incorporate discontinuous jumps we now start with new stock dynamics in the Merton Jump Diffusion framework.

Assumption 6.11. *The real world and risk neutral distributions for the jump component are the same.*

Definition 6.12. The Merton Jump Diffusion Model has the stock dynamics

$$(6.3) \quad dS(t) = (r - K\lambda)S(t)dt + \sigma S(t)dW^*(t) + (y_i - 1) dN(t).$$

Where $N(t)$ is a Poisson process with intensity λ , $W^*(t)$ is a Wiener process in the risk neutral world and $K = e^{\mu + \frac{\sigma^2}{2}} - 1$.

This model combines the usual Black-Scholes diffusion with sudden jumps. The correction term $-K\lambda$ ensures the process stays risk-neutral — a kind of “expected jump drag.” We have incorporated the existence of a jump but we now need to define a jump multiplier to see how big each jump is.

Definition 6.13. We will call the fractional size of the discontinuous jump the jump stock process $S(t)$ as y_i where i corresponds to the jump number with the formula:

$$y_i = \frac{S(t+)}{S(t-)},$$

denoting $S(t-)$ denotes the left-hand limit of S at time t , and $S(t+)$ denoting the right- hand limit of S at time t .

Assumption 6.14. $y_1, y_2, \dots, y_i$ are independent and identically distributed with log normally distribution such that

$$\ln y_i \sim N(\mu_{y_i}, \sigma_{y_i}^2).$$

$$(6.4) \quad y_i \sim \text{Lognormal} \left(\exp \left(\mu_{y_i} + \frac{\sigma_{y_i}^2}{2} \right), \exp (2\mu_{y_i} + \sigma_{y_i}^2) (\exp (\sigma_{y_i}^2) - 1) \right).$$

Finally we derive the solution for the stock value under Merton Jump Diffusion. This will be the stock value we shall use in the calculations to derive the prices for the down and out Barrier option.

Theorem 6.15. [6] *The solution to the dynamics Equation (6.3) is*

$$(6.5) \quad S(t) = S(0)e^{(\mu - K\lambda - \frac{1}{2}\sigma^2)t + \sigma W(t) + \sum_{i=1}^{N(t)} \ln y_i}.$$

Proof. Taking dynamics equation to be of form (6.3), let $F : \mathbb{R} \rightarrow \mathbb{R}$ to be $F(x) = \ln(x)$ and $x = S(t)$ using Theorem 6.10 we deduce the following:

$$\begin{aligned} d\ln(S(t)) &= \left(\frac{r - K\lambda}{S(t)} S(t) - \frac{\frac{1}{2}\sigma^2 S^2(t)}{S^2(t)} \right) dt + \frac{\sigma S(t)}{S(t)} dW(t) \\ &\quad + (\ln y + \ln(S(t)) - \ln(S(t))) dN(t), \\ &= \left(r - K\lambda - \frac{1}{2}\sigma^2 \right) dt + \sigma dW(t) + \ln y dN(t). \end{aligned}$$

Hence

$$\ln S(t) = \ln S(0) + \int_0^t \left(r - K\lambda - \frac{1}{2}\sigma^2 \right) dt + \int_0^t \sigma dW(t) + \sum_{i=1}^{N(t)} \ln y_i.$$

Which leads to

$$\frac{S(t)}{S(0)} = \exp \left(\int_0^t \left(r - K\lambda - \frac{1}{2}\sigma^2 \right) dt + \int_0^t \sigma dW(t) + \sum_{i=1}^{N(t)} \ln y_i \right).$$

So

$$\begin{aligned} S(t) &= S(0) \exp \left(\int_0^t r - K\lambda - \frac{1}{2}\sigma^2 dt + \int_0^t \sigma dW(t) + \sum_{i=1}^{N(t)} \ln y_i \right), \\ &= S(0) \exp \left((r - K\lambda - \frac{1}{2}\sigma^2)t + \sigma W(t) + \sum_{i=1}^{N(t)} \ln y_i \right). \end{aligned}$$

□

From now on we will use the notation

$$x(t) = \ln S(t).$$

Theorem 6.16. [6] *In the Merton Jump Diffusion framework, let the logarithmic form of the stock price, $S(t)$, be denoted as $x(t) = \ln S(t)$ then*

$$x(T) - x(0) \sim N \left(\left(r - \frac{\sigma^2}{2} - \lambda k \right) T + \mu_y \lambda T, \sigma^2 T + [\mu_y^2 + \sigma_y^2] \lambda T \right).$$

Proof. From the solution of the Merton Jump Diffusion SDE, the stock price at time T is given by eq(6.5) :

$$S(T) = S(0) \exp \left((r - K\lambda - \frac{1}{2}\sigma^2)t + \sigma W(t) + \sum_{i=1}^{N(t)} \ln y_i \right).$$

Taking logarithms and setting $x(t) = \ln S(t)$, we obtain

$$x(T) = \ln(S(0)) + (r - K\lambda - \frac{1}{2}\sigma^2)t + \sigma W(t) + \sum_{i=1}^{N(t)} \ln y_i.$$

Subtracting $x(0) = \ln(S(0))$, we get:

$$x(T) - x(0) = (r - K\lambda - \frac{1}{2}\sigma^2)t + \sigma W(T) + \sum_{i=1}^{N(t)} \ln y_i.$$

We now compute the expectation:

$$\mathbb{E}(x(T) - x(0)) = (r - K\lambda - \frac{1}{2}\sigma^2)t + \mathbb{E}(\sigma(W(T) - W(0))) + \mathbb{E}\left(\sum_{i=1}^{N(t)} \ln y_i\right).$$

Since $\mathbb{E}[W(T)] = 0$ and $\mathbb{E}\left(\sum_{i=1}^{N(t)} \ln y_i\right) = \lambda T \mathbb{E}(y_i) = \lambda T \mu_y$, it follows that:

$$\mathbb{E}(x(T) - x(0)) = (r - K\lambda - \frac{1}{2}\sigma^2)T + \mu_y \lambda T.$$

Next, we compute the variance since the Brownian part and jump part are independent:

$$\text{Var}(x(T) - x(0)) = \text{Var}\left((r - K\lambda - \frac{1}{2}\sigma^2)t + \sigma(W(T) - W(0))\right) + \text{Var}\left(\sum_{i=1}^{N(t)} \ln y_i\right).$$

We know that:

$$\text{Var}[\sigma W(T)] = \sigma^2 T,$$

and using properties of compound Poisson processes from Theorem 6.5, we have:

$$\text{Var}\left(\sum_{i=1}^{N(t)} \ln y_i\right) = (\mu_y + \sigma_y) \lambda T.$$

Hence, the increment $x(T) - x(0)$ is normally distributed as:

$$x(T) - x(0) \sim N\left(\left(r - \frac{\sigma^2}{2} - \lambda k\right)T + \mu_y \lambda T, \sigma^2 T + [\mu_y^2 + \sigma_y^2] \lambda T\right).$$

□

The algorithm we employ calculates the stock's dynamics by separately handling the continuous diffusion and the jump components. Specifically, we use Brownian bridges, which we will define in the next section, to model the deterministic behavior of stock price movements. This approach offers the advantage of isolating the predictable deterministic movements from the stochastic jumps, streamlining the calculation of stock price paths.

Under this framework, the pure diffusion component of the log-return $x(T) - x(0)$ follows a normal distribution:

$$(6.6) \quad x(T) - x(0) \sim N\left(\left(r - \frac{\sigma^2}{2} - \lambda k\right)T, \sigma^2 T\right).$$

From now on we will use the notation $c = r - K\lambda - \frac{1}{2}\sigma^2$ for simplicity of notation.

Then future use we can also use the fact

$$(6.7) \quad x(T) \sim N(x(0) + cT, \sigma^2 T).$$

This will be useful for computing the endpoint of the Brownian bridge in the algorithm used. The Brownian bridge ensures that the diffusion is interpolated between known points in a consistent manner, while the jump dynamics are incorporated separately to account for the discrete, stochastic impacts on the stock price.

By treating the continuous diffusion and discontinuous jumps independently, the algorithm simplifies path generation and provides a clear method for modelling the Merton Jump Diffusion dynamics. The algorithms used shall be explained later on.

Theorem 6.17. *The Logarithmic Stock Price $x(t)$ follows the dynamics:*

$$dx(t) = cdt + \sigma dW^*(t),$$

where $W^*(t)$ is a Wiener Process in the Risk-Neutral world.

We have defined the stock process under the risk-neutral measure. To avoid confusion, we shall use P to denote the risk-neutral measure when calculating probabilities related to the stock from now on.

7. OPTION PRICE CALCULATION

To figure out the option's payoff in our algorithm, we'll finally bring in the Brownian bridge — yep, that mysterious thing we've been hinting at. Time to give it a proper introduction!

Definition 7.1. Let $W(t)$ be a Wiener Process defined in the probability space $(\Omega, \mathcal{B}, \mathbb{P})$, a standard Brownian bridge, $B(t)$ over the interval $[0, T]$ is a stochastic process given by:

$$B(t) = W(t) - \frac{t}{T}W(T). \text{ for } 0 \leq t < T,$$

Definition 7.2. A Brownian bridge $B^{(a,b)}$ over the time interval $[0, T]$, that starts at a and ends at b is given by:

$$B^{(a,b)} = a + (b - a)\frac{t}{T} + B(t), \text{ for } 0 \leq t < T,$$

where $B(t)$ is a standard Brownian bridge defined over the interval $[0, T]$ and $a, b \in \mathbb{R}$.

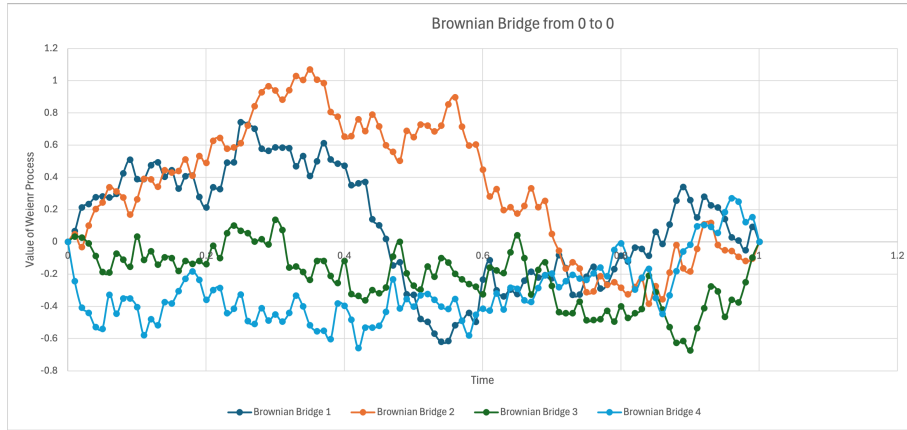


FIGURE 7.1. Example of paths on a Brownian bridge, $B(t)$.

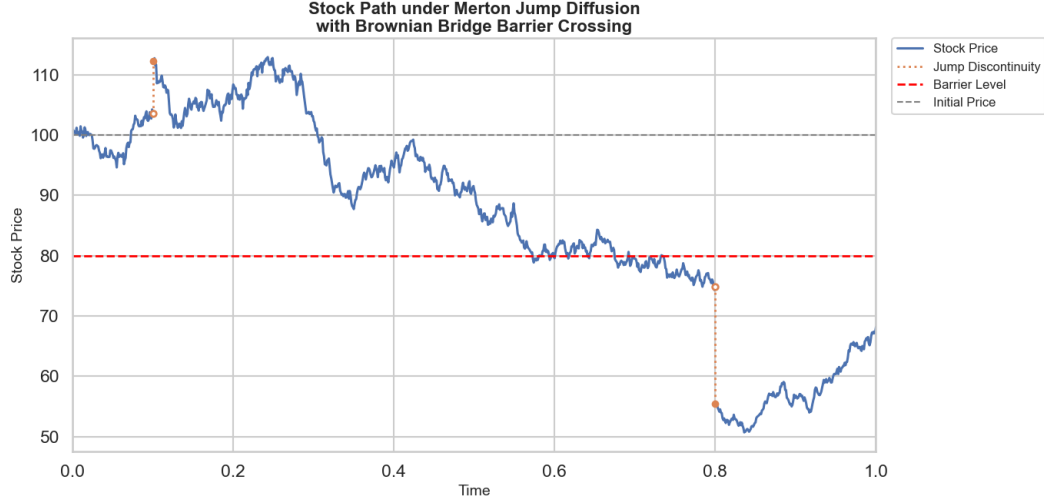


FIGURE 7.2. Example of stock path simulation under Merton Jump Diffusion dynamics.

This diagram depicts a trajectory of $S(t)$ where discontinuous jumps are marked by dotted lines, and jump times are explicitly indicated. The jump times, denoted as τ_i follow an exponential distribution with intensity λ , representing the time interval from the $(i-1)^{th}$ jump until the i^{th} jump. Here there are two jumps occurring at 0.1 and 0.8. The corresponding jump sizes, follow the distribution given in Equation (6.4). The unshaded orange endpoints of the Brownian bridge, denoted as T_i are independently simulated using Equation (6.6).

Since each Brownian bridge we simulate is assumed to be independent, we can compute the probability of crossing a barrier separately for each bridge. This motivates the definition of a density function for the first crossing time, which allows us to avoid simulating every point in a Monte Carlo simulation. To formalise this, we define below the first crossing event as C and the corresponding first crossing probability density as $g_i(t)$. To streamline the following calculations, we adopt the following notation for the i^{th} jump:

$$x(T_i^+) := \lim_{h \rightarrow T_i^+} x(h),$$

and

$$x(T_i^-) := x(T_i-) = \lim_{h \rightarrow T_i^-} x(h).$$

Definition 7.3. Let $S(t)$ be a stock price process and $x(t) = \ln S(t)$ as its logarithm. We can define the event C as the first crossing of a barrier H at time t , given that $x(a) > \ln H$ for all $a \in [0, t]$. Formally this is,

$$C = \{t : \ln S(t) = \ln H \text{ and } \ln S(a) > \ln H \text{ for all } a \in [0, t]\}.$$

We can now formalise the conditioned density of a barrier crossing.

Definition 7.4. Let C be defined as in Definition 7.3, and let

$$x(t) = \ln S(t),$$

denote the logarithm of the stock price process $S(t)$. We shall denote $g_i(t)$ as the probability that the first crossing for the process crosses barrier H , given the end-points of $x(T_i^-)$ and $x(T_{i-1}^+)$, occurs within $[T_{i-1}, T_i]$:

$$(7.1) \quad g_i(t) = P(C \in [T_{i-1}, T_i] \mid x(T_i^-) = a, x(T_{i-1}^+) = b).$$

In this section, we focus specifically on pricing down-and-out call options. There are 3 scenarios which can happen with stock $S(t)$ in each Brownian bridge and the discounted payoff can be evaluated by considering these cases:

- (1) A Brownian bridge crossing occurs before the jump

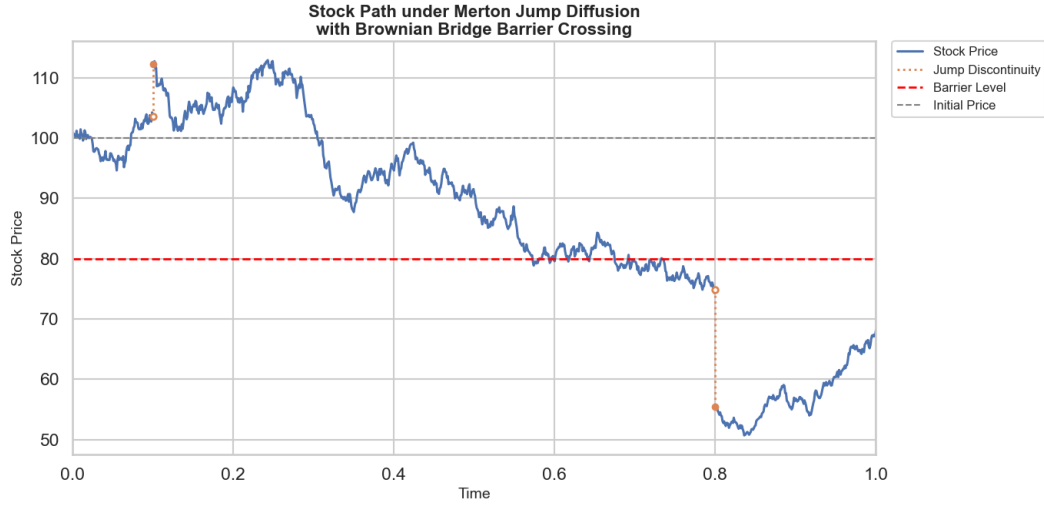


FIGURE 7.3. Stock path under Merton Jump Diffusion where first crossing occurs during the continuous Brownian bridge.

- (2) The barrier crossing occurs after the jump

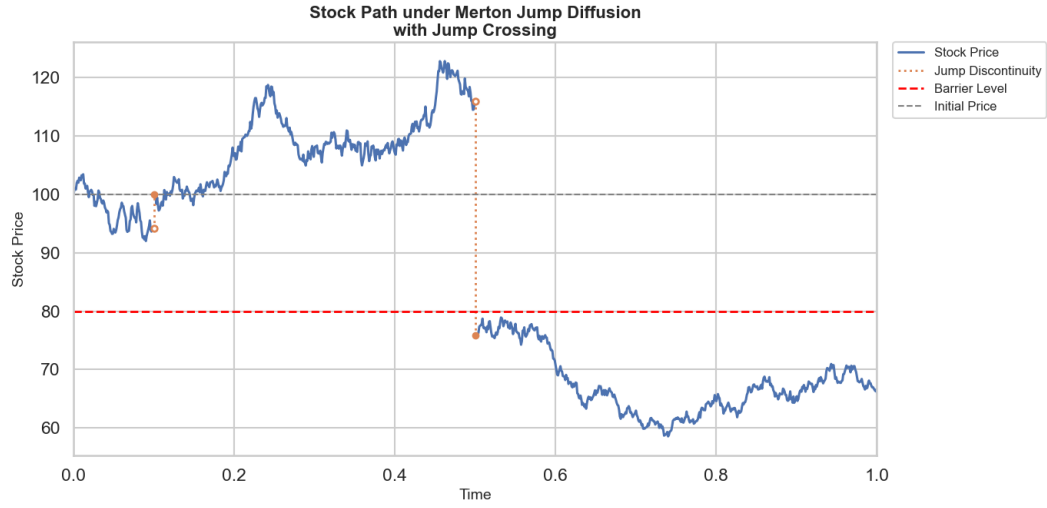


FIGURE 7.4. Stock path under Merton Jump Diffusion where first crossing occurs during a jump.

(3) There is no crossing during the bridge

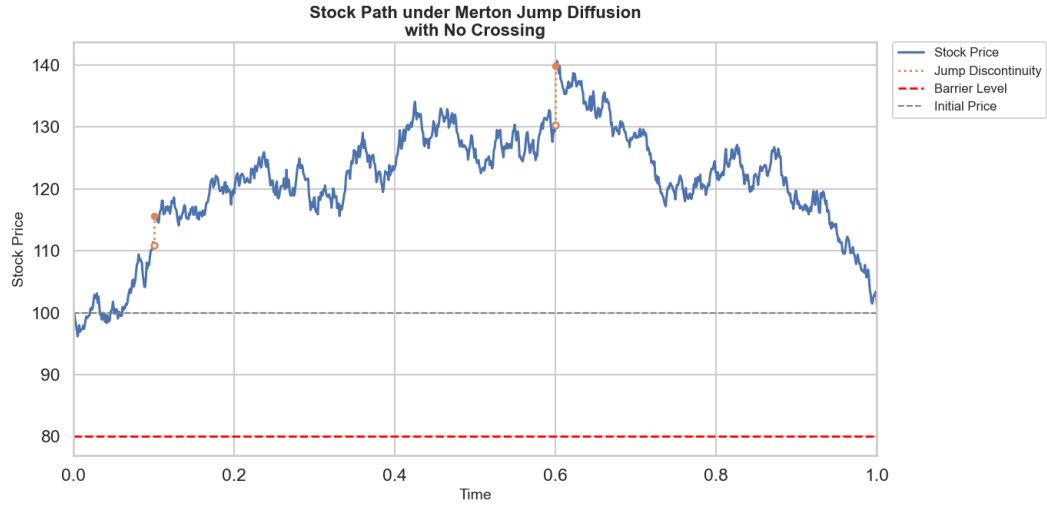


FIGURE 7.5. Stock path under Merton Jump Diffusion where no crossing occurs.

Let

$$I = \min\{i : S(T_i) \leq H\},$$

and assume there are $K + 1$ jumps.

Define

$$U = \begin{cases} I & \text{if } I \neq 0, \\ K + 1 & \text{Otherwise.} \end{cases}$$

In the initial scenario Figure 7.3, the expected payoff can be expressed as

$$\sum_{i=1}^U R \int_{T_{i-1}}^{T_i} \exp(-rs) g_i(s) ds,$$

with probability

$$\prod_{j=1}^{i-1} P_j.$$

Here $g_i(s)$ denotes the conditional density for the process to hit the barrier within the interval $[T_{i-1}, T_i]$, given the Brownian bridge between the two jump times. The rebate R represents the payoff received upon a barrier crossing. For each interval between consecutive jumps, we evaluate the probability of barrier breach under the Brownian bridge approximation, discount the corresponding payoff, and compute the total expected value accordingly.

For Figure 7.4 the discounted payoff becomes

$$\exp(-rT_I) R,$$

with probability

$$\prod_{j=1}^I P_j,$$

as the payoff would be the rebate R and it is weighted by the probability of there not being a crossing in all previous Brownian bridges.

For Figure 7.5, if this occurs for all of the bridges then the discounted payoff is given by

$$\exp(-rT) \times \max(\exp(x(T)) - K, 0),$$

where the probability of this event is

$$\prod_{j=1}^{K+1} P_j,$$

due to the independence of the Brownian bridges, as here the barrier reaches expiry and has the payoff of a call option in this case.

Now we have an idea of the payoff and how the Bridges work lets formally define a Brownian bridge for the stock process.

Definition 7.5. Consider a stock process $S(t)$ defined in Merton Jump Diffusion framework (6.3), a Brownian bridge on the interval $[t_0, T]$, denoted as $B_{(a,b)}$, is a process conditioned on the logarithmic stock process $x(t) = \ln S(t)$ such that $x(t_0) = a$ and $x(T) = b$ where $a, b \in \mathbb{R}$.

To see if the stock has crossed the during barrier, this is equivalent at looking at the minimum stock value in the bridge. As the Brownian bridge is a continuous process if the minimum stock value is below the barrier then there has been a crossing. Below is the density that there has not been a crossing. To calculate

the density of Equation (7.1) we shall split the conditional density up into smaller densities that we can calculate, which we will then link them together in the proof.

Theorem 7.6. [9] *Let $S(t)$ represent the stock's value at time t and define the running infimum of a stock's value over the time $[s, t]$ as*

$$m(u) = \inf_{s \leq u \leq t} \ln(S(u)).$$

The probability that the $m(t)$ remains above a given barrier H given the start and endpoint of the process is

$$(7.2) \quad \begin{aligned} p_i &= P(m(u) > \ln H \mid x(s) = c, x(t) = d), \\ &= \begin{cases} 1 - \exp\left(-\frac{2[c-a][c-b]}{\tau\sigma^2}\right) & \text{if } b > c, \\ 0 & \text{Otherwise.} \end{cases} \end{aligned}$$

Where $x(u) = S(u)$ for $u \in [s, t]$.

Proof. As we are conditioning on the start of the process we can call this $S(0)$ informally so we can rewrite the equation as, taking continuous part of Equation (6.5)

$$S(t) = S(0)e^{(r-K\lambda-\frac{1}{2}\sigma^2)t+\sigma W(t)}.$$

All components besides $W(t)$ are deterministic and strictly increasing in t any decrease in $S(t)$ must result from a decrease in the Wiener process $W(t)$. As the exponential function is strictly increasing, the minimum of $S(t)$ occurs precisely when $W(t)$ attains its minimum. This justifies that the asset price and the Wiener process reach their minima simultaneously, in the absence of jumps. By manipulating Equation (6.5) let's write the endpoint of a new Weiner Process $W^*(T)$ in P^* as:

$$W(T) + \Theta T = a,$$

where

$$a = \frac{\frac{x(t)}{x(s)}}{\sigma}, \quad \Theta = \frac{r - K\lambda}{\sigma} - \frac{\sigma}{2}, \quad b = \frac{\ln \frac{H}{x(s)}}{\sigma} \quad \text{and} \quad T = t - s.$$

We rewrite

$$(7.3) \quad \begin{aligned} P(m(u) > \ln H \mid x(s) = c, x(t) = d) &= P\left(\inf_{s \leq u \leq t} x(u) > \ln H \mid x(s) = c, x(t) = d\right), \\ &= P\left(\inf_{a \leq t \leq b} (W(t) + \Theta t) < b \mid \right. \\ &\quad \left. W(0) = 0, W(t_1) + \Theta T = a\right). \end{aligned}$$

The condition $x(t) = c$ corresponds to $W(0) = 0$ since $x(t)$ represents the starting point of a Brownian bridge. We also assume that each Brownian bridge is independent of all others.

$$P\left(\inf_{a \leq t \leq b} W(t) + \Theta t < b \mid W(0) = 0, W(t_1) + \Theta T = a\right) = \mathbb{E}^P(1_{\inf_{a \leq t \leq b} W(t) + \Theta t < b} \mid W(0) = 0, W(t_1) + \Theta T = a).$$

Take $W^*(t) = W(t) + \Theta t$, using Girsanov's Theorem we can change measure

$$\frac{dP^*}{dP} = \exp \left\{ -\Theta W(t) - \frac{\sigma^2 T}{2} \right\} = \exp \left\{ \Theta W^*(t) - \frac{\sigma^2 T}{2} \right\}.$$

Using Bayes Rule we can expand Equation (7.3) and incorporate Girsanov's Theorem to get:

$$\begin{aligned} &= \frac{\mathbb{E}^{P^*} (\beta 1_{\inf_{a \leq t \leq b} W^*(t) > b} \mid W(0) = 0, W(t_1) + \theta T = a)}{\mathbb{E}^{P^*} (\beta \mid W(0) = 0, W(t_1) + \theta T = a)}, \\ &= \mathbb{E}^{P^*} (1_{\inf_{a \leq t \leq b} W^*(t) > b} \mid W(0) = 0, W(t_1) + \theta T = a), \\ &= P^* \left(\inf_{a \leq t \leq b} W^*(t) > b \mid W^*(0) = 0, W^*(t_1) = a \right). \end{aligned}$$

Using Lemma 5.5 and Corollary 5.7 we can solve the above probability

$$\begin{aligned} P^* \left(\inf_{a \leq t \leq b} W^*(t) > b \mid W^*(0) = 0, W^*(t_1) = a \right) &= 1 - \exp \left(\frac{a^2 - (2b - a)^2}{2T} \right), \\ &= 1 - \exp \left(\frac{(4b^2 - 4ab)}{2T} \right), \\ &= 1 - \exp \left(\frac{4b(b - a)}{2T} \right), \\ &= 1 - \exp \left(\frac{4b(b - a)}{2T} \right). \end{aligned}$$

Substituting for a , b and T we get the desired result. $\square$

Below is the C++ implementation of the no-crossing probability in a Brownian bridge, corresponding to equation (7.2). This function returns the probability that the stock path, conditioned on values at times t_1 and t_2 remains above the barrier H throughout the interval $[t_1, t_2]$, under the Merton Jump Diffusion model without accounting for jumps during this period.

```
long double DownAndOut::NoCrossingDensity(MJD stock, double A, double
    B, double t1, double t2){ //Probability of stock not crossing
    in the brownian bridge

    double sigma = stock.GetSigma();
    double tau = t2 - t1;

    if (B > std::log(H)) {
        double ExpTerm = (2.0 * (std::log(H) - A) * (std::log(H) - B)
            ) / (tau * sigma * sigma);
        return 1.0 - std::exp(-ExpTerm);
    }
    else {
        return 0.0;
    }
}
```

We now need to find the first stopping time given a starting value of the logarithmic stock.

Lemma 7.7. [8] Let C be defined as in Definition 7.3 such that

$$C := \{x(s) = \ln H \text{ for some } s \in (T_{i-1}, T_i]\}.$$

Then the probability density of the first hitting time at $\tau_H := \inf \{s > T_{i-1} : x(s) = \ln H\}$ given $x(T_{i-1}^+) = a$ is

$$(7.4) \quad P(\tau_H \in [T_{i-1}, t] | x(T_{i-1}^+) = a) = \int_{T_{i-1}^+}^t \frac{a - \ln H}{\sqrt{2\pi}\sigma} (t - T_{i-1})^{-\frac{3}{2}} \times \\ \exp\left(-\frac{(a - \ln H + c(t - T_{i-1}))^2}{2(t - T_{i-1})\sigma^2}\right) dt.$$

Where $\ln H$ is the natural logarithm of barrier H , σ is the volatility of the stock process, $c = r - K\lambda - \frac{1}{2}\sigma^2$ and a is the natural logarithm of the stock.

Proof. The process evolves as

$$x(t) = a + ct + \sigma W(t).$$

Define $\tau = \inf \{s > 0 : x(s) = \ln H\}$. So now our objective is to compute the conditional probability

$$P(\tau \in [t, t + dt] | x(0) = a).$$

To proceed, we apply Girsanov's Theorem to eliminate the drift c . Define a new probability measure P^* : under which the process

$$W^*(t) := W(t) + \frac{c}{\sigma}t,$$

is a standard Brownian motion. The Radon-Nikodym derivative of P^* with respect to P

$$\frac{dP^*}{dP} = \exp\left(\frac{c}{\sigma}W^*(t) - \frac{c^2}{\sigma^2}t\right).$$

Under which the process $x(t)$ becomes:

$$x(t) = a + \sigma W^*(t).$$

So hitting $\ln H$ means

$$W^*(t) = \frac{a - \ln H}{\sigma}.$$

We multiplied the above by negative 1, this is valid as $-W(t)$ is a Wiener Process so we treat this as a Wiener process from Lemma 5.8:

$$P(R_c \in [t, t + \delta]) = \int_t^{t+\delta} \frac{c}{\sqrt{2\pi}t^3} \exp\left(-\frac{c^2}{2t}\right) dt.$$

Then under change of measure our desired result becomes:

$$P(\tau_H \in [T_{i-1}^+, t] | x(T_{i-1}^+) = a) = \mathbb{E}^{P^*} \left[\frac{dP^*}{dP} 1_{R_c \in [t, t+\delta]} \right].$$

Letting $c = \frac{a - \ln H}{\sigma}$:

$$P(\tau_H \in [T_{i-1}^+, t] | x(T_{i-1}^+) = a) = \int_{T_{i-1}^+}^t \frac{a - \ln H}{\sigma \sqrt{2\pi t^3}} \exp\left(-\frac{(a - \ln H)^2}{2t\sigma^2} + \frac{c}{\sigma} \left(\frac{a - \ln H}{\sigma}\right) - \frac{c^2}{\sigma^2} t\right) dt.$$

By collecting like terms and completing the square we get the desired result:

$$P(\tau_H \in [T_{i-1}^+, t] | x(T_{i-1}^+) = a) = \int_{T_{i-1}^+}^t \frac{a - \ln H}{\sigma \sqrt{2\pi t^3}} \exp\left(\frac{-(a - \ln H) + ct)^2}{2t\sigma^2}\right) dt.$$

The density that the logarithmic stock value lies within a certain value given the starting point is derived below. $\square$

Lemma 7.8. [8] *The probability that $x(T_i^-)$ lies within the interval $[b, b + \delta]$, where $\delta > 0$ and small, is given the initial value after the previous jump $x(T_{i-1}^+) = a$, is given by:*

$$(7.5) \quad P(x(T_i^-) \in [b, b + \delta] | x(T_{i-1}^+) = a) = \int_b^{b+\delta} \frac{1}{\sqrt{2\pi(T_i - T_{i-1})}\sigma} \times \exp\left(-\frac{(b - a - c(T_i - T_{i-1}))^2}{2(T_i - T_{i-1})\sigma^2}\right) db.$$

Proof. From definition of the stock dynamics, the stock equation is :

$$S(T_i^-) = S(T_{i-1}^+) e^{(r - K\lambda - \frac{1}{2}\sigma^2)t + \sigma W(t)},$$

$$x(T_i^-) = \ln S(t).$$

Using $x(T_{i-1}^+) = a$ we can rewrite as

$$x(T_i^-) = a + ct + \sigma W^*(t).$$

Since $W^*(t) \sim N(0, 1)$ we can conclude that:

$$x(T_i^-) \sim N(a + ct, \sigma^2 t).$$

Hence the probability that $x(T_i^-) \in [b, b + \delta]$ given an arbitrary start point is:

$$P(x(T_i^-) \in [b, b + \delta] | x(T_{i-1}^+) = a) = \int_b^{b+\delta} \frac{1}{\sqrt{2\pi(T_i - T_{i-1})}\sigma} \times \exp\left(-\frac{(b - a - c(T_i - T_{i-1}))^2}{2(T_i - T_{i-1})\sigma^2}\right) db.$$

The density that the logarithmic stock value lies within a certain value given there has been a crossing is derived below. $\square$

Lemma 7.9. [8] *The probability that $x(T_i^-)$ lies within the interval $[b, b + \delta]$ given $x(t) = \ln H$, is given by:*

$$(7.6) \quad P(x(T_i^-) \in [b, b + \delta] | x(t) = \ln H) = \int_b^{b+\delta} \frac{1}{\sqrt{2\pi(T_i - t)}\sigma} \times \exp\left(-\frac{(b - \ln H - c(T_i - t))^2}{2(T_i - t)\sigma^2}\right) dt.$$

Proof. From Equation 6.3 the stock equation is :

$$\begin{aligned} S(T_i^-) &= S(T_{i-1}^+)e^{(r-K\lambda-\frac{1}{2}\sigma^2)t+\sigma W(t)}, \\ x(T_i^-) &= \ln S(t). \end{aligned}$$

Using $x(t) = \ln H$ we can rewrite as

$$x(T_i^-) = \ln H + ct + \sigma W^*(t).$$

Since $W^*(t) \sim N(0, 1)$ we can conclude that:

$$x(T_i^-) \sim N(\ln H + ct, \sigma^2 t).$$

Hence the probability that $x(T_i^-) \in [b, b + \delta]$ given an arbitrary start point is:

$$\begin{aligned} P(x(T_i^-) \in [b, b + \delta] | x(t) = \ln H) &= \int_b^{b+\delta} \frac{1}{\sqrt{2\pi(T_i - t)}\sigma} \\ &\times \exp\left(-\frac{(b - \ln H - c(T_i - t))^2}{2(T_i - t)\sigma^2}\right) db. \end{aligned}$$

□

Remark 7.10. Using Definition 2.11, we will use these shorthand expressions derived from the equations to simplify notation and avoid rewriting the full equations repeatedly:

- (1) The conditional density $f_{(x(T_i^-)|x(t))}(b | \ln H)$ is equivalent to the expression given in Equation (7.6).
- (2) The conditional density $f_{(x(T_i^-)|x(T_{i-1}^+)=b)}(b | a)$ is equivalent to the expression given in Equation (7.5).
- (3) The conditional density $f_{(C|x(T_{i-1}^+))}(t | a)$ is equivalent to the expression given in Equation (7.4).

We finally have the building blocks to derive a density for Equation (7.1)!

Theorem 7.11. [8] *Let $g_i(t)$ be as defined equation (7.1). Then the solution is given by:*

$$\begin{aligned} (7.7) \quad g_i(t) &= \frac{x(T_{i-1}^+) - \ln H}{2\gamma\pi\sigma^2} (t - T_{i-1})^{-\frac{3}{2}} (T_i - t)^{-\frac{1}{2}} \times \\ &\exp\left(-\left(\frac{(x(T_i^-) - \ln H - c(T_i - t))^2}{2(T_i - t)\sigma^2} + \frac{(x(T_{i-1}^+) - \ln H + c(t - T_{i-1}))^2}{2(t - T_{i-1})\sigma^2}\right)\right), \end{aligned}$$

where

$$\gamma = \frac{1}{\sqrt{2\pi(T_i - T_{i-1})}\sigma} \times \exp\left(-\frac{(x(T_i^-) - x(T_{i-1}^+) - c(T_i - T_{i-1}))^2}{2(T_i - T_{i-1})\sigma^2}\right).$$

Proof. For ease of notation lets denote:

Using Definition 2.10 we can express

$$f_{(C, x(T_i^-), x(T_{i-1}^+))}(t, a, b),$$

as the joint probability density function of the random variables $C, x(T_i^-) = a, x(T_{i-1}^+) = b$. Similarly, we denote

$$f_{(x(T_i^-), x(T_{i-1}^+))}(a, b),$$

as the joint probability density function of the random variables $x(T_i^-)$ and $x(T_{i-1}^+)$ and using Definition 2.11 we can denote

$$f_{(C|x(T_i^-), x(T_{i-1}^+))}(t | a, b),$$

as the joint probability density function of the random variables $C, x(T_i^-) = a, x(T_{i-1}^+) = b$.

Using equation (7.1) we can rewrite $g_i(t)$ as:

$$\begin{aligned} g_i(t) &= f_{(C|x(T_i^-), x(T_{i-1}^+))}(t | a, b), \\ &= \frac{f_{(C, x(T_i^-), x(T_{i-1}^+))}(t, a, b)}{f_{(x(T_i^-), x(T_{i-1}^+))}(a, b)}. \end{aligned}$$

We can simplify the numerator using Bayes' theorem:

$$f_{(C, x(T_i^-), x(T_{i-1}^+))}(t, a, b) = f_{(x(T_i^-)|C=t, x(T_{i-1}^+))}(b | t, a) \times f_{(C|x(T_{i-1}^+))}(t | a) \times f_{(x(T_{i-1}^+))}(a).$$

Similarly we can simplify the denominator :

$$f_{(x(T_i^-), x(T_{i-1}^+))}(a, b) = f_{(x(T_i^-)|x(T_{i-1}^+))}(b | a) \times f_{(x(T_{i-1}^+))}(a).$$

Combining to get:

$$g_i(t) = \frac{f_{(x(T_i^-)|C=t, x(T_{i-1}^+))}(b | t, a) \times f_{(C|x(T_{i-1}^+))}(t | a)}{f_{(x(T_i^-)|x(T_{i-1}^+)=b)}(b | a)}.$$

Applying the Markov property from Theorem 2.21 we get

$$f_{(x(T_i^-)|C=t, x(T_{i-1}^+))}(b | t, a) = f_{(x(T_i^-)|x(t))}(b | \ln H) \text{ for all } t \in [T_{i-1}, T_i].$$

Leaving

$$g_i(t) = \frac{f_{(x(T_i^-)|x(t))}(b | \ln H) \times f_{(C|x(T_{i-1}^+))}(t | a)}{f_{(x(T_i^-)|x(T_{i-1}^+)=b)}(b | a)}.$$

From Remark 7.10 we can use the densities of the results, which were proved earlier to get:

$$\begin{aligned} g_i(t) &= \frac{\frac{1}{\sqrt{2\pi(T_i-t)}\sigma} \times \exp\left(-\frac{(b-a-c(T_i-T_{i-1}))^2}{2(T_i-T_{i-1})\sigma^2}\right)}{\frac{1}{\sqrt{2\pi(T_i-T_{i-1})}\sigma} \times \exp\left(-\frac{(b-a-c(T_i-T_{i-1}))^2}{2(T_i-T_{i-1})\sigma^2}\right)} \\ &\quad \times \frac{a - \ln H}{\sqrt{2\pi}\sigma} (t - T_{i-1})^{-\frac{3}{2}} \times \exp\left(-\frac{(a - \ln H + c(t - T_{i-1}))^2}{2(t - T_{i-1})\sigma^2}\right). \end{aligned}$$

Then using simple algebra and letting:

$$\gamma = \frac{1}{\sqrt{2\pi(T_i - T_{i-1})}\sigma} \times \exp\left(-\frac{(x(T_i^-) - x(T_{i-1}^+) - c(T_i - T_{i-1}))^2}{2(T_i - T_{i-1})\sigma^2}\right),$$

we get the desired result. $\square$

Below is the C++ implementation of the first-passage time density for a Brownian bridge, corresponding to equation (7.7). The `evaluate_gi()` function computes the probability density of the stock path crossing the barrier H for the first time at time t , conditioned on log-prices a and b at times T_1 and T_2 . The auxiliary function `gamma()` calculates the transition density of the Brownian bridge, adjusted for drift c , under the Merton Jump Diffusion model.

```
double DownAndOut::evaluate_gi(MJD stock, double a, double b, double
    t, double T1, double T2) { //Density of Crossing for the first
    time during the Brownian Bridge
    double c = stock.GetC();
    double sigma = stock.GetSigma();
    double gamma_val = gamma(stock,a,b,T1, T2);
    double section1 = ((a - std::log(H)) / (2 * gamma_val * M_PI *
        sigma * sigma))
        * std::pow(t - T1, -3.0/2.0)
        * std::pow(T2 - t, -1.0/2.0);

    double expTerm1 = (std::pow((b - std::log(H) - c * (T2 - t)),
        2.0))
        / (2 * (T2 - t) * sigma * sigma);

    double expTerm2 = (std::pow((a - std::log(H) + c * (t - T1)),
        2.0))
        / (2 * (t - T1) * sigma * sigma);

    return section1 * std::exp(-(expTerm1 + expTerm2));
}

double DownAndOut::gamma(MJD stock, double a, double b, double T1,
    double T2) {
    double c = stock.GetC();
    double sigma = stock.GetSigma();
    return (1.0 / (std::sqrt(2 * M_PI * (T2 - T1)) * sigma))
        * std::exp(-(std::pow((a - b) + c * (T2 - T1), 2.0))
        / (2 * sigma * sigma * (T2 - T1)));
}
```

8. ALGORITHM FOR PRICING CALL BARRIER OPTION

8.1. Setup. "We now arrive at the implementation stage of the algorithms. Two methods will be presented for pricing a down-and-out barrier call option: one based on uniform sampling, and the other using a Taylor series approximation of the first-passage density. The algorithms for computing the payoffs will be detailed, and to estimate the option price, one simply repeats the simulation and averages the results over the number of paths."

We first need to be able to simulate the stock path and subsequent jumps that occur. For both methods, it is necessary to retrieve the jump times, which are modelled as exponentially distributed random variables. as they are often sufficient

for modelling the timing of random events. Below is the C++ implementation of the jump time simulation for the Merton Jump Diffusion model. The function **JumpTimes()** generates jump times over the interval $[0, 1]$ where inter-arrival times are sampled from an exponential distribution with rate parameter λ . The resulting vector includes the start time 0 and end time 1, marking the boundaries of the interval.

```
std::vector<double> MJD::JumpTimes() { //returns the Jump times that
    are exponentially distributed between [0,1]
    std::vector<double> Times;

    std::exponential_distribution<> exp_dis(lambda);

    double count = 0.0;
    Times.push_back(count);
    while (count <= 1) {
        double time = exp_dis(RandomGenerator::getGenerator());
        if (count + time > 1) break; // Stop if exceeding 1
        count += time;
        Times.push_back(count); // Store cumulative jump time
    }
    Times.push_back(1.0);
    return Times;
}
```

Additionally, we must determine the jump sizes, which are derived from the distribution specified in Equation (6.4). Below we have C++ implementation of jump size generation under the Merton Jump Diffusion model. The **JumpDynamics()** function returns a jump size sampled from a normal distribution with mean μ_J and standard deviation σ_J representing the logarithmic jump magnitude.

```
double JumpSize::JumpDynamics(){ //returns the Jump Size using
    normal distribution
    std::normal_distribution <> d(JumpMu,JumpSigma);
    return d(RandomGenerator::getGenerator());
}
```

To simulate the continuous part of the process, we construct a Brownian bridge using the initial stock price and the dynamics defined in Equation (6.7). Below we have the C++ implementation of the continuous (diffusive) component of the Merton Jump Diffusion model. The **ContinuousDynamics()** function simulates the log-price evolution over the interval $[t_1, t_2]$, using a normal distribution with drift-adjusted mean and volatility-scaled standard deviation, conditioned on the starting value **Start**.

```
double MJD::ContinuousDynamics(double Start , double t1, double t2){

    assert(t2 > t1 && "times are incorrect");
    double time = t2 - t1;
    double mean = Start + c * time;
    double stddev = sigma * std::sqrt(time);
```

```

std::normal_distribution<> d{mean, stddev};
double generate = d(RandomGenerator::getGenerator());
return generate;
}

```

Now we can separate into the different methods.

8.2. Uniform Sampling algorithm. A high level view of what is happening is that we are taking a uniform sample with the probability of there not being a crossing, we can do this by altering the interval we are sampling from.

Following on from the initial steps outlined above, we first compute Equation (7.2), which is labelled P_i in the code, this gives the probability that the minimum stock price remains above the barrier during the interval $[T_{i-1}, T_i]$. This allows us to calculate the extension of the interval:

$$\text{Extension of Interval} = \frac{T_{i-1}, T_i}{1 - p_i}.$$

Next, we generate a uniform random variable s from $[T_{i-1}, T_{i-1} + E]$, where E is the extension length defined above.

We then follow the steps below:

Step 1: Brownian bridge check

If $s \in [T_{i-1}, T_i]$ there has been a crossing during the Brownian bridge:

- Evaluate $g_i(s)$ using Equation (7.1) and calculate the discounted payoff:

$$\text{DiscPayoff}(n) = Rg_i(s) \exp(-rs), \text{ where } R \text{ is the rebate.}$$

– Exit the function.

Step 2: Jump event check

If a crossing did not occur during the Brownian bridge, update the stock price after the jump by adding the jump size to the log price:

$$x(T_i^+) = x(T_i^-) + \text{Jump Size},$$

- (1) If $x(T_i^+) \leq \ln H$ then there has been a crossing during the jump so the payoff is

$$\text{DiscPayoff}(n) = \exp(-r \times T_i^+) \times R,$$

- Exit the function.

- (2) If $x(T_i^+) > \ln H$ then we can repeat the above for each Jump Time until option expiry.

Step 3 : No barrier crossing

If no barrier crossing occurs before the option's expiration T , the option is valid and the discounted payoff is:

$$\text{DiscPayoff}(n) = R \exp(-rT) \times \max(\exp(x(T)) - X, 0).$$

Below we have the C++ implementation of the Uniform Sampling algorithm.

```

double Barrier::PriceByMJD_Uniform(MJD stock){
    std::vector<double> Times;
    Times = stock.JumpTimes(); //generates exponentially distributed
                               jump times
    double StockPriceAfterJump = stock.GetLogS0();
    int i = 0;
}

```

```

bool Checker = 1;
double StockPriceBeforeJump = 0.0;
while(i+1 < Times.size()){
    StockPriceBeforeJump = stock.ContinuousDynamics(
        StockPriceAfterJump,Times[i],Times[i+1]); //returns stock
        value at the end of the continous interval

    double SizeOfJump = stock.GetJumpDynamics();
    long double P_i = NoCrossingDensity(stock , StockPriceAfterJump
        , StockPriceBeforeJump,Times[i],Times[i+1] );
    double ExtentionOfInterval = (Times[i+1]- Times[i]) / (1.0-P_i)
        ;
    std::uniform_real_distribution <> d{Times[i], Times[i]+
        ExtentionOfInterval};
    double Sample = d(RandomGenerator::getGenerator());
    assert(Sample > Times[i] && "Invalid time of sample ");

    if(Sample < Times[i+1] ) //if there is a crossing during the
        bridge
    {
        double Payoff = evaluate_gi(stock,StockPriceAfterJump,
            StockPriceBeforeJump, Sample,Times[i],Times[i+1] )
            * std::exp(-stock.GetRF() * Sample) *
            Rebate * ExtentionOfInterval;

        Checker = 0;

        return Payoff;
    }

    if(i + 2 < Times.size()){
        StockPriceAfterJump = StockPriceBeforeJump + SizeOfJump ;
    }

    if(StockPriceAfterJump <= std::log(H)) //if there is a crossing
        during the jump
    {
        double Payoff = std::exp( - stock.GetRF() * Times[i+1]) *
            Rebate;
        Checker = 0;

        return Payoff;
    }

    i++;

}

if(Checker){ //if there is no crossing for the entire lifespan
    of the option
```

```

    double TerminalValue = std::exp(StockPriceBeforeJump);

    return Rebate * std::exp(- stock.GetRF() ) * Payoff(
        TerminalValue) ;
}
}

```

8.3. Taylor Series Approximation. An overview of this method is that instead of running separate paths we approximate the probability of there being a crossing for each bridge, this is by approximating the integral from equation (8.1), which is derived in the appendix. As we walk through the algorithm, we will reference the code below to explain each step in detail.

We now repeat the setup process to generate the Jump Times, JumpSizes and construct the first Brownian bridge. Throughout the simulation, we denote the discounted payoff by DiscPayoff use multiplyPi to represent the ongoing product of the P_i calculations.

Initialization:

Set DiscPayoff(n)=0.

Computation of P_i :

Evaluate P_i for the first Brownian bridge using Equation 7.2, which gives the probability that the minimum of the process stays above the barrier.

Evaluation of the integral $J(s)$

Compute the integral using approximation from Equation (12) [The partial derivation is given in the appendix] :

$$(8.1) \quad J(s) = \int_{T_{i-1}}^{T_i} \exp(-rs) g_i(s) ds,$$

where $g_i(s)$ is Equation (7.1).

Update of the discounted payoff :

$$\text{DiscPayoff}(n) = \text{DiscPayoff}(n) + RJ(s) \prod_{j=1}^{i-1} P_j.$$

This represent the discounted payoff of there being an initial cross in the current Brownian bridge, note that $\prod_{j=1}^{i-1} P_j = 1$ when $i = 1$ as there is no crossing in the previous bridge as this is the first bridge.

If $\text{StockPriceBeforeJump} \leq \ln H$ then return DiscPayoff(n). If this is not the case then we need to update

$$\text{StockPriceAfterJump} = \text{StockPriceBeforeJump} + \text{JumpSize}.$$

If $\text{StockPriceAfterJump} \leq \ln H$ then there has been a crossing during a jump so the updated discounted payoff is

$$\text{DiscPayoff}(n) = \text{DiscPayoff}(n) + R \exp(-rT_i) \prod_{j=1}^i P_j.$$

If $\text{StockPriceAfterJump} > \ln H$ then examine the next interval and repeat. If no crossing has occurred for the life cycle :

$$\text{DiscPayoff}(n) = \text{DiscPayoff}(n) + \exp(-rT) \prod_{j=1}^{K+1} P_j \times \max(\exp(x(T)) - X, 0).$$

When $n = N$, all cycles of the Monte Carlo simulation have been completed, compute the estimate for the option price as:

$$DOC = \frac{1}{N} \sum_{n=1}^N \text{DiscPayoff}(n).$$

We are essentially following this payoff structure from [8] :

Let DOC represent the discounted expectation of the down and out barrier option, then

$$\begin{aligned} DOC = & \sum_{i=1}^U R \prod_{j=1}^{i-1} P_j \int_{T_{i-1}}^{T_i} \exp(-rs) g_i(s) ds + \\ & A_I \exp(-rT_I) R \prod_{j=1}^I P_j + \\ & (1 - A_I) \exp(-rT) \times \max(\exp(x(T)) - X, 0) \prod_{j=1}^{K+1} P_j. \end{aligned}$$

Where I is the index of first jump that violates the barrier condition, if the stock does not cross the barrier then $I = 0$. We have the following definitions corresponding to the variables above:

$$A_I = \begin{cases} 1 & \text{if } I \neq 0, \\ 0 & \text{if } I = 0, \end{cases}$$

and

$$U = \begin{cases} I & \text{if } I \neq 0, \\ K + 1 & \text{if } I = 0. \end{cases}$$

The probability of no crossing in within period $[T_{j-1}, T_j]$ is P_j and R is the rebate paid if the option goes out of the money. Below we have the C++ implementation of Taylor Method algorithm.

```
double Barrier::PriceByMJD_Taylor(MJD stock){

    std::vector<double> Times;
    Times = stock.JumpTimes(); //generates exponentially distributed
                               jump times
    double Pay = 0;
    ModelParams p;
    p.r = stock.GetRF();
    p.sigma = stock.GetSigma();
    p.LogBarrier = std::log(H);
    double StockPriceAfterJump = stock.GetLogS0();
    int i = 0;
    bool Checker = 1;
```

```

double StockPriceBeforeJump = 0.0;
double multiplyPi = 1;
while(i+1 < Times.size()){
    StockPriceBeforeJump = stock.ContinuousDynamics(
        StockPriceAfterJump, Times[i], Times[i+1]); //returns stock
        value at the end of the continous interval
    double SizeOfJump = stock.GetJumpDynamics();
    long double P_i = NoCrossingDensity(stock ,
        StockPriceAfterJump, StockPriceBeforeJump, Times[i], Times[i
        +1] ); //Probability that there is no corssing during the
        brownian bridge
    p.T1 = Times[i];
    p.T2 = Times[i+1];
    p.X1 = StockPriceAfterJump;
    p.X2 = StockPriceBeforeJump;
    double J = EstimateGI(p);
    //here we are going to check this:
    // long double Intergral_Check = Call_trapezium(stock,
        StockPriceAfterJump, StockPriceBeforeJump, Times[i],
        Times[i+1]);

    // std::cout << "Here is the difference: " << std::abs(J-
        Intergral_Check) << std::endl;
    Pay = Pay + Rebate * J * multiplyPi;
    if(i + 2 < Times.size()){
        StockPriceAfterJump = StockPriceBeforeJump + SizeOfJump ;
    }
    multiplyPi = multiplyPi * P_i;
    if(StockPriceBeforeJump <= std::log(H)){ //if there is a
        crossing during the bridge
        Checker = 0;
        return Pay;
        //there should be a return here surely?
    }
    else if(StockPriceAfterJump <= std::log(H)){ //if there is a
        crossing during the jump
        Checker = 0;

        Pay = Pay + Rebate * std::exp(-stock.GetRF() * Times[i+1]) *
            multiplyPi;
    }
    i++;
}

if( Checker){ //if there is no crossing for the entire lifespan
    of the option
    double TerminalValue = std::exp(StockPriceBeforeJump);

```

```

        return Pay + multiplyPi * Payoff(TerminalValue) * std::exp(-
            stock.GetRF());
    }
}

```

9. EXTENSION OF UNIFORM ALGORITHM

To improve the efficiency of the Monte Carlo simulation, variance control techniques are employed. Specifically, we make use of a control variate whose expected value is known and is close to that of the barrier option. This helps reduce the variance of the estimator without introducing bias.

In the context of our implementation using the uniform sampling algorithm, we modify the simulation output to return a vector rather than a scalar. This allows us to retain the full path of the underlying asset and compute the call option price at maturity alongside the barrier option price. This extension enables a straightforward incorporation of the control variate method.

Definition 9.1. The variance reduction is achieved using the following control variate formula, we shall define the control variate estimator as:

$$Z = X - \beta(Y - \tilde{Y}),$$

where X is the payoff of the option, Y is the payoff of the control variate, $\tilde{Y}$ is the expected value of the payoff of the control variate, which is known, and $\beta \in \mathbb{R}$.

Theorem 9.2. [2] *Let Y be a control variate such that $\mathbb{E}(Y) = \tilde{Y}$ is known. Define the control variate estimator as. Let Z be a control variate estimator of form*

$$Z = X - \beta(Y - \tilde{Y}),$$

for some $\beta \in \mathbb{R}$. Then the β that minimizes the variance of Z is given by:

$$\beta^* = \frac{\text{Cov}(X, Y)}{\text{Var}(Y)}.$$

Proof. The Variance of Z can be written as:

$$\text{Var}(Z) = \text{Var}(X - \beta(Y - \tilde{Y})).$$

$\tilde{Y}$ is known and constant, as Variance is unaffected by addition and subtraction of constant we can rewrite

$$\text{Var}(Z) = \text{Var}(X - \beta Y).$$

We now apply the standard formula for the variance of a linear combination of random variables:

$$\text{Var}(X - \beta Y) = \text{Var}(X) - 2\beta \text{Cov}(X, Y) + \beta^2 \text{Var}(Y).$$

To minimize $Var(Z)$, we differentiate with respect to β and set equal to 0 to find an optimal solution

$$\begin{aligned}\frac{\delta}{\delta\beta} [Var(Z)] &= \frac{\delta}{\delta\beta} [Var(X - \beta Y)], \\ 0 &= -2Cov(X, Y) + 2\beta Var(Y),\end{aligned}$$

Solving for β yields the optimal coefficient

$$\beta = \frac{Cov(X, Y)}{Var(Y)}.$$

□

For the control variate we will use a European call option under Merton Jump Dynamics, in 1976, Robert Merton derived a semi-closed form solution for the valuation of European-style options in a market where the underlying asset price follows a jump-diffusion process with normally distributed jump sizes.

Theorem 9.3. [7] *Let $C(S_0, T)$ denote the price of a European option under the Merton Jump Diffusion model with constant jump intensity λ , such that the distribution of the jump J is $J \sim N(\mu_J, \sigma_J)$, maturity T and initial asset price $S(0)$. Then the option price can be expressed as a weighted sum of Black-Scholes prices:*

$$C(S(0), T) = \sum_{n=0}^{\infty} \frac{\exp(-\lambda T) \times (\lambda T)^n}{n!} \times C_{BS}(S_0, T, r_n, \sigma_n, K),$$

such that

$$\sigma_n = \sqrt{\sigma^2 + \frac{n\sigma_J^2}{T}},$$

$$r_n = r - \lambda(\mu_J - 1) + n \log \frac{\mu_J}{T},$$

$$\lambda' = \lambda\mu_J,$$

and $C_{BS}(S_0, T, r_n, \sigma_n, K)$ is the value of the call option when calculated in the Black Scholes Model framework.

We shall omit the proof and showing the value of the call option under the Black Scholes Model framework, but refer to [3] for the Black Scholes derivation. Below is the C++ implementation.

```
double norm_cdf(double x) {
    return 0.5 * std::erfc(-x / std::sqrt(2));
}

// Black-Scholes formula for a European call option
double black_scholes_call(double S, double K, double T, double r,
    double sigma) {
    double d1 = (std::log(S / K) + (r + 0.5 * sigma * sigma) * T) /
        (sigma * std::sqrt(T));
    double d2 = d1 - sigma * std::sqrt(T);

    return S * norm_cdf(d1) - K * std::exp(-r * T) * norm_cdf(d2);
}
```

```

double PriceMJD(MJD stock, int N, double Strike) {
    double price = 0.0;

    // Extract parameters
    double S0 = stock.GetS0();
    double muJ = stock.GetJumpMu(); // Mean of log jump size
    double sigJ = stock.GetJumpSig(); // Stddev of log jump size
    double sigma = stock.GetSigma(); // Diffusion volatility
    double r = stock.GetRF(); // Risk-free rate
    double lambda = stock.GetLamda(); // Jump intensity (expected #
        jumps per year)
    double T = 1.0; // Time to maturity in years

    // Compute kappa = E[Y - 1], where Y = e^Z is the jump
        multiplier
    double kappa = exp(muJ + 0.5 * sigJ * sigJ) - 1.0;

    // Loop over number of jumps
    for (int n = 0; n < N; ++n) {
        // Adjust volatility and drift for n jumps
        double sigma_n = std::sqrt(sigma * sigma + (n * sigJ * sigJ)
            / T);
        double r_n = r - lambda * kappa + (n * (muJ + 0.5 * sigJ *
            sigJ)) / T;

        // Poisson probability of n jumps in time T
        double poisson_prob = exp(-lambda * T) * std::pow(lambda * T,
            n) / std::tgamma(n + 1.0);

        // Black-Scholes price for adjusted parameters
        price += poisson_prob * black_scholes_call(S0, Strike, T, r_n,
            sigma_n);
    }

    return price;
}

```

Below is the Uniform sampling algorithm with the control variate European Call under Merton Jump Dynamics:

```

std::vector<double>Barrier::UniformVarRedCall(MJD stock){
    std::vector<double> Times;
    Times = stock.JumpTimes(); //generates exponentially distributed
        jump times
    double StockPriceAfterJump = stock.GetLogS0();
    int i = 0;
    bool Checker = 1;
    double StockPriceBeforeJump = 0.0;
    double BarrierPay = 0.0;

```

```

while(i+1 < Times.size()){
    StockPriceBeforeJump = stock.ContinuousDynamics(
        StockPriceAfterJump,Times[i],Times[i+1]); //returns stock
        value at the end of the continous interval

    double SizeOfJump = stock.GetJumpDynamics();
    long double P_i = NoCrossingDensity(stock , StockPriceAfterJump
        , StockPriceBeforeJump,Times[i],Times[i+1] );
    double ExtentionOfInterval = (Times[i+1]- Times[i]) / (1.0-P_i)
        ;
    std::uniform_real_distribution <> d{Times[i], Times[i]+
        ExtentionOfInterval};
    double Sample = d(RandomGenerator::getGenerator());
    assert(Sample > Times[i] && "Invalid time of sample ");

    if(Sample < Times[i+1] ) //if there is a crossing during the
        bridge
    {
        if(BarrierPay == 0.0){
            BarrierPay = evaluate_gi(stock,StockPriceAfterJump,
                StockPriceBeforeJump, Sample,Times[i],Times[i+1] )
                * std::exp(-stock.GetRF() * Sample) *
                Rebate * ExtentionOfInterval;
        }
        Checker = 0;
    }

    if(i + 2 < Times.size()){
        StockPriceAfterJump = StockPriceBeforeJump + SizeOfJump ;
    }

    if(StockPriceAfterJump <= std::log(H)) //if there is a crossing
        during the jump
    {
        if(BarrierPay == 0.0){

            BarrierPay = std::exp( - stock.GetRF() * Times[i+1]) * Rebate
                ;

        }
        Checker = 0;
    }

    i++;
}

```

```

if(Checker){ //if there is no crossing for the entire lifespan
              of the option
    double TerminalValue = std::exp(StockPriceBeforeJump);
    BarrierPay = Rebate * std::exp(- stock.GetRF() ) * Payoff(
        TerminalValue);
    double Callpay = Payoff(TerminalValue) * std::exp(-stock.
        GetRF());

    return {BarrierPay,Callpay} ;
}
else{double TerminalValue = std::exp(StockPriceBeforeJump);
    double Callpay = Payoff(TerminalValue) * std::exp(-stock.
        GetRF());
    return {BarrierPay,Callpay };}
}

```

10. NUMERICAL RESULTS

Now to see if all this work was worth it, here are the results obtained from running the Monte Carlo simulation with one million paths for each test case. The analysis focuses on the estimated option price, its associated standard deviation, and the total computation time in milliseconds.

The following three test cases were used to evaluate the performance of both algorithms:

- (1) $S(0) = 50, K = 55, H = 45, R = 1, r = 0.05, \sigma = 0.3, \lambda = 8, \sigma_y = 0.05, \mu_y = 0$, and $T = 1$.
- (2) $S(0) = 100, K = 110, H = 95, R = 1, r = 0.05, \sigma = 0.25, \lambda = 2, \sigma_y = 0.1, \mu_y = 0$, and $T = 1$.
- (3) $S(0) = 100, K = 110, H = 85, R = 1, r = 0.05, \sigma = 0.25, \lambda = 2, \sigma_y = 0.1, \mu_y = 0$, and $T = 1$.

Method	Time (milliseconds)	Standard Error
Uniform Sampling	10149	0.00989149
Taylor Series Approximation	11783	0.00941559
Standard Monte Carlo	154001	0.00993936
Uniform Sampling with Control	16215	0.00544927

TABLE 1. Experiment 1 (True Value 4.513).

Method	Time (milliseconds)	Standard Error
Uniform Sampling	5502	0.0145775
Taylor Series Approximation	5327	0.0119681
Standard Monte Carlo	60591	0.0151725
Uniform Sampling with Control	7332	0.0103122

TABLE 2. Experiment 2 (True Value 5.303).

Method	Time (milliseconds)	Standard Error
Uniform Sampling	5380	0.0180751
Taylor Series Approximation	6133	0.01759
Standard Monte Carlo	87156	0.0180605
Uniform Sampling with Control	8131	0.00548228

TABLE 3. Experiment 3 (True Value 9.013).

The results across all three experiments demonstrate a clear trade-off between computational efficiency and estimation accuracy among the four methods evaluated: Uniform Sampling, Taylor Series Approximation, Standard Monte Carlo, and Uniform Sampling with Control Variate. Notably, the Uniform Sampling with Control Variate method consistently produced the lowest standard error, indicating the highest level of precision. In the first experiment, this method achieved a standard error of 0.00545—almost half that of the other methods—while requiring only 16,215 ms, a reasonable computational cost considering the accuracy gained. Similarly, in the second and third experiments, it maintained the lowest standard errors at 0.01031 and 0.00548 respectively, again outperforming the other approaches in terms of precision. Although it was not the fastest method, the reduction in standard error was significant enough to justify the modest increase in computation time.

In contrast, Uniform Sampling without control variates was consistently the quickest method, particularly in Experiments 2 and 3 (5,502 ms and 5,380 ms respectively), but this came at the expense of higher standard errors, ranging from 0.0146 to 0.0181. The Taylor Series Approximation method offered a favourable balance between speed and accuracy, especially in the second experiment where it was the fastest (5,327 ms) and more accurate than standard Uniform Sampling. However, its performance was less impressive in the other two experiments. Standard Monte Carlo, despite being a classical and widely used technique, performed poorly in terms of computational efficiency. It recorded the highest runtimes in all three experiments—most notably 154,001 milliseconds in the first—and did not provide any substantial improvement in accuracy, with standard errors that were comparable to or worse than those of the faster methods. These findings highlight the effectiveness of control variates in improving estimation precision without incurring excessive computational costs. They also suggest that while classical methods like Standard Monte Carlo may be conceptually robust, their practical application is often limited by inefficiency.

Overall, Uniform Sampling with Control Variate emerges as the most effective method, striking a strong balance between accuracy and computational feasibility for the problem studied.

It is worth noting that the Standard Monte Carlo method used time steps of 0.01 and incorporated jumps by simulating the jump times, running the Monte Carlo over each interval, and then calculating the corresponding jump sizes—repeating this process until expiry. While it would have been ideal to experiment with a smaller time step to improve accuracy, practical limitations intervened. The code was executed on a rather ageing MacBook Air with an Intel chip, approaching its eighth birthday. When attempting to run the simulation with finer granularity, the computation time exceeded five minutes, and the device began to emit concerning

levels of heat—prompting fears it might quite literally overheat. As such, results for smaller time steps were regrettably omitted in the interest of both efficiency and laptop safety.

To assess the convergence behaviour of the standard error for each method, we plotted the logarithm of the standard deviation against the logarithm of the number of simulations as shown in Figure 10.1. The gradients of the fitted plots are shown below:

Method	Gradient
Uniform Sampling	−1.98294464
Taylor Series Approximation	−1.992575003
Uniform Sampling with Control	−1.811986286

TABLE 4. Gradients of convergence in each method.

This approach allows us to estimate the convergence rate by calculating the gradient of the resulting line, with a gradient close to -1 indicating convergence at the theoretical rate for Monte Carlo methods. The gradients obtained were -1.9829 for Uniform Sampling, -1.9926 for the Taylor Series Approximation, and -1.8120 for Uniform Sampling with Control Variate. Both Uniform Sampling and the Taylor method exhibit gradients very close to -2 , suggesting they converge slightly faster than the standard theoretical rate. This is likely due to the relatively smooth structure of the problem and the variance reduction effects implicitly present in the sampling techniques. In contrast, the gradient for Uniform Sampling with Control Variate is slightly shallower at -1.8120 .

While this still indicates good convergence, it suggests that the addition of the control variate affects the scaling of the variance reduction as the number of simulations increases. Nevertheless, despite a slightly lower convergence gradient, the control variate method consistently achieved lower standard errors across all simulation sizes, implying that it remains the most efficient method in practice when balancing both accuracy and computational effort.

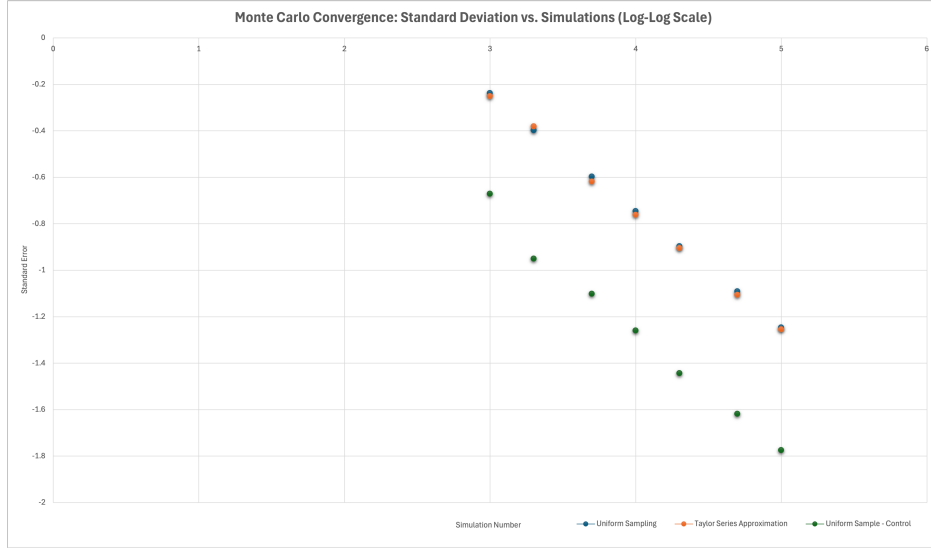


FIGURE 10.1. Log-Log scale of the convergence of standard deviation as the number of simulations increase.

The standard error is calculated using the unbiased estimator as discussed in [2]

$$S_N^2 = \frac{1}{N-1} \sum_{i=1}^N (H_i - \mu_N)^2,$$

where H_i is the payoff in the i^{th} simulation, N is the total number of simulations and μ_N is the true value of the option, which is given in [8]. The standard deviation is calculated by

$$\sigma = \frac{S_N}{\sqrt{N}}.$$

11. CONCLUSION

The numerical results obtained throughout this study firmly confirm the accuracy, efficiency, and robustness of the proposed Monte Carlo algorithms for pricing down-and-out barrier options within the framework of the Merton Jump Diffusion model. Across all three distinct test cases considered, the estimators consistently produced option price estimates that closely aligned with theoretical or benchmark values, thereby validating the correctness and reliability of the algorithmic implementations. This strong agreement between computed results and expected values demonstrates that the numerical methods are capable of capturing the complex dynamics inherent in jump-diffusion processes, which are known to pose significant challenges for traditional pricing techniques.

Among the different methods examined, the Taylor Series Approximation emerged as a particularly effective approach, consistently yielding slightly lower standard errors across the experiments compared with the Uniform Sampling and Standard Monte Carlo algorithms. This suggests that it achieves greater estimator precision without compromising computational speed, striking a commendable balance

between accuracy and efficiency. However, the Uniform Sampling with Control Variate method proved to be especially noteworthy. Despite incurring a moderate increase in computation time relative to the simpler Uniform Sampling technique, it achieved significantly the lowest standard errors in every experiment. This substantial improvement highlights the practical benefits of incorporating control variates as a variance reduction technique, especially in the context of jump-diffusion models where the stochastic nature of jumps can introduce considerable noise into the simulation results.

It is important to emphasise that the numerical framework adopted in this work assumes the barrier is positioned substantially below the initial stock price. This assumption is crucial because the control variate employed is based on a standard Call option, and its effectiveness diminishes if the barrier is located close to the initial asset value. When the barrier is near the starting price, the likelihood of barrier breach increases, complicating the estimation process and reducing the efficacy of the control variate. By situating the barrier well below the initial price, the method leverages the relatively low probability of hitting the barrier, which enables more accurate conditioning and variance reduction.

As anticipated, increasing the jump intensity within the model led to longer computation times. This arises from the requirement to perform additional Brownian bridge evaluations between jump occurrences to accurately simulate the path behaviour. While this increases the computational burden compared to the pure diffusion case (where jumps are absent), the additional cost is justified by the ability of the Merton Jump Diffusion model to more realistically capture sudden discontinuities and jumps in asset prices, phenomena that are well documented in financial markets but are neglected in simpler models.

The convergence analysis further substantiates the strength and reliability of the proposed methods. Empirical convergence rates were observed to be markedly faster than the classical Monte Carlo convergence rate of -0.5 , with the Taylor Series Approximation and Uniform Sampling (with and without a control variate) methods exhibiting gradients close to -2 . This notably rapid convergence rate underscores the effectiveness of the conditioning techniques and variance reduction strategies implemented within the algorithms, enhancing the computational efficiency without sacrificing accuracy. Such performance improvements are particularly valuable in practical applications, where computational resources and time constraints are critical considerations.

In summary, the Monte Carlo schemes developed and tested in this research—especially the variant enhanced with control variates—offer a highly effective and theoretically sound framework for pricing exotic options in markets characterised by jump risk. These methods deliver a compelling combination of precision, speed, and robustness, making them well-suited for real-world financial engineering problems. Looking ahead, future research could build upon these findings by exploring hedging strategies for jump-diffusion derivatives in incomplete markets, where the absence of a unique equivalent martingale measure renders perfect replication impossible. Such extensions would involve quantifying hedging errors and investigating optimal risk-minimising strategies under conditions of model uncertainty, thereby providing deeper insights and practical tools for portfolio managers and risk professionals tasked with managing exposure to discontinuous price movements.

12. APPENDIX

Lets prove the lemmas from the prelims before getting onto the Taylor expansion:

Lemma 12.1. *The Laplace transform of the function $x(t) = at^{-\frac{3}{2}} \exp\left(\frac{-a^2}{4t}\right)$ is given by:*

$$\mathcal{L}\left\{at^{-\frac{3}{2}} \exp\left(\frac{-a^2}{4t}\right)\right\} = 2\sqrt{\pi} \exp(-a\sqrt{s}).$$

Proof. First we shall use substitution of $b = a^2$, we get:

$$\begin{aligned} \mathcal{L}\left[at^{-\frac{3}{2}} \exp\left(\frac{-a^2}{4t}\right)\right] &= \mathcal{L}\left[b^{\frac{1}{2}}t^{-\frac{3}{2}} \exp\left(\frac{-b}{4t}\right)\right], \\ &= b^{\frac{1}{2}}\mathcal{L}\left[t^{-\frac{3}{2}} \exp\left(\frac{-b}{4t}\right)\right]. \end{aligned}$$

Using equation (2.1):

$$\begin{aligned} b^{\frac{1}{2}}\mathcal{L}\left[t^{-\frac{3}{2}} \exp\left(\frac{-b}{4t}\right)\right] &= b^{\frac{1}{2}}2\sqrt{\pi}b^{-\frac{1}{2}} \exp\left(\sqrt{-bs}\right), \\ &= 2\sqrt{\pi} \exp(-a\sqrt{s}). \end{aligned}$$

□

Lemma 12.2. *The Laplace transform of the function $x(t) = t^{-\frac{1}{2}} \exp\left(\frac{-a^2}{4t}\right)$ is given by:*

$$\mathcal{L}\left[t^{-\frac{1}{2}} \exp\left(\frac{-a^2}{4t}\right)\right] = \frac{\sqrt{\pi} \exp(-|a|\sqrt{s})}{\sqrt{s}}.$$

Proof. First we shall use substitution of $b = a^2$, we get:

$$\mathcal{L}\left[t^{-\frac{1}{2}} \exp\left(\frac{-a^2}{4t}\right)\right] = \mathcal{L}\left[t^{-\frac{1}{2}} \exp\left(\frac{-b}{4t}\right)\right].$$

Using equation (2.2):

$$\begin{aligned} \mathcal{L}\left[t^{-\frac{1}{2}} \exp\left(\frac{-b}{4t}\right)\right] &= \sqrt{\pi}s^{-\frac{1}{2}} \exp\left(\sqrt{-bs}\right), \\ &= \frac{\sqrt{\pi} \exp(-|a|\sqrt{s})}{\sqrt{s}}. \end{aligned}$$

□

For the proposed algorithm in Subsection 8.3 we are using Taylor series we want to estimate:

$$J(s) = \int_{T_{i-1}}^{T_i} \exp(-rs) g_i(s) ds.$$

To achieve this, we will use the a Taylor expansion on a Laplace transform and leverage its convolution and time-shifting properties. To rewrite the integral in a form suitable for applying the Convolution Theorem 2.18, we perform the substitution $s = t + T_{i-1}$. This gives:

$$\int_{T_{i-1}}^{T_i} \exp(-rs) g_i(s) ds = \int_0^{T_i - T_{i-1}} \exp(-r(t + T_{i-1})) \hat{g}_i(t) dt,$$

where $\tau = T_i - T_{i-1}$ and $g_i(\hat{t}) = g_i(T_{i-1} + t)$ is the time shifted version of $g_i(t)$. Simplifying further:

$$\int_0^{T_i - T_{i-1}} \exp(-r(t + T_{i-1})) g_i(\hat{t}) dt = \exp(-rT_{i-1}) \int_0^\tau \exp(-rs) g_i(\hat{s}) ds.$$

Let $J(\tau)$ represent the integral:

$$J(\tau) = \exp(-rT_{i-1}) \int_0^\tau \exp(-rs) g_i(\hat{s}) ds.$$

We can rewrite $g_i(\hat{t})$ in terms of two function $f(t)$ and $h(t)$:

$$J(\tau) = \frac{\exp(-rT_{i-1})}{\gamma} \int_0^\tau \exp(-rs) f(t) h(\tau - t) ds,$$

where $f(t)$ and $h(t)$ are defined as:

$$f(t) = \frac{(x(T_{i-1}^+) - \ln H)}{\sqrt{2\pi\sigma^2}} t^{-\frac{3}{2}} \exp\left(-\frac{(x(T_{i-1}^+) - \ln H + ct)^2}{2t\sigma^2}\right),$$

$$h(t) = \frac{\exp(-rt)}{\sqrt{2\pi\sigma^2}} t^{-\frac{1}{2}} \exp\left(-\frac{(x(T_i^-) - \ln H - c(T_i - t))^2}{2(T_i - t)\sigma^2}\right).$$

The integral is now expressed as a convolution:

$$J(\tau) = \exp(-rT_{i-1}) \times (\text{convolution of } f(t) \text{ and } h(t)).$$

By the Theorem 2.18, we can find $J(\tau)$ more efficiently by determining the Laplace transforms of $f(t)$ and $h(t)$ and multiplying them. We now need to find the Laplace

transform for $f(t)$:

$$\begin{aligned}
\mathcal{L}[f(t)] &= \mathcal{L} \left[\frac{(x(T_{i-1}^+) - \ln H)}{\sqrt{2\pi\sigma^2}} t^{-\frac{3}{2}} \exp \left(-\frac{(x(T_{i-1}^+) - \ln H + ct)^2}{2t\sigma^2} \right) \right], \\
&= \frac{1}{\sqrt{2\pi}} \mathcal{L} \left[\left(\frac{x(T_{i-1}^+) - \ln H}{\sigma^2} \right) t^{-\frac{3}{2}} \exp \left(-\frac{2(x(T_{i-1}^+) - \ln H + ct)^2}{4t\sigma^2} \right) \right], \\
&= \frac{1}{\sqrt{2\pi}} \mathcal{L} \left[\frac{x(T_{i-1}^+) - \ln H}{\sigma^2} t^{-\frac{3}{2}} \right] \times \\
&\quad \exp \left(\frac{-2(x(T_{i-1}^+) - \ln H)^2 - 4ct(x(T_{i-1}^+) - \ln H) - 2c^2t^2}{4t\sigma^2} \right), \\
&= \frac{1}{\sqrt{2\pi}} \exp \left(\frac{c(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \mathcal{L} \left[\frac{x(T_{i-1}^+) - \ln H}{\sigma^2} t^{-\frac{3}{2}} \right] \\
&\quad \times \exp \left(\frac{-2(x(T_{i-1}^+) - \ln H)^2 - c^2t}{4t\sigma^2} \right), \\
&= \frac{1}{\sqrt{2\pi}} \frac{\sqrt{2}}{\sqrt{2}} \exp \left(\frac{c(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \mathcal{L} \left[\frac{x(T_{i-1}^+) - \ln H}{\sigma^2} t^{-\frac{3}{2}} \right] \\
&\quad \times \exp \left(\frac{-2(x(T_{i-1}^+) - \ln H)^2 - c^2t}{4t\sigma^2} \right), \\
&= \frac{1}{2\sqrt{\pi}} \exp \left(\frac{c(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \mathcal{L} \left[\frac{\sqrt{2}(x(T_{i-1}^+) - \ln H)}{\sigma^2} t^{-\frac{3}{2}} \right] \\
&\quad \times \exp \left(\frac{-2(x(T_{i-1}^+) - \ln H)^2 - c^2t}{4t\sigma^2} \right).
\end{aligned}$$

Using Equation (2.3) where we shall take

$$a = \frac{\sqrt{2}(x(T_{i-1}^+) - \ln H)}{\sigma}.$$

We can carry on the above cancellation:

$$\begin{aligned}
&= \frac{2\sqrt{\pi}}{2\sqrt{\pi}} \exp \left(\frac{c(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \\
&\quad \times \exp \left(-\frac{\sqrt{2}(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \sqrt{s + \frac{c^2}{2\sigma^2}}, \\
&= \exp \left(\frac{c(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \exp \left(-\frac{\sqrt{2}(x(T_{i-1}^+) - \ln H)}{\sigma} \right) \\
&\quad \times \sqrt{s + \frac{c^2}{2\sigma^2}}.
\end{aligned}$$

Now calculating the Laplace transform of $h(t)$:

$$\begin{aligned}
\mathcal{L}[h(t)] &= \mathcal{L} \left[\frac{1}{\sqrt{2\pi\sigma^2}} t^{-\frac{1}{2}} \exp \left(-\frac{2(x(T_i^-) - \ln H - c(T_i - t))^2}{4(T_i - t)\sigma^2} \right) \right], \\
&= \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ \frac{c(x(T_i^-) - \ln H)}{\sigma^2} \right\} \mathcal{L} \left[t^{-\frac{1}{2}} \exp \left(\frac{-2(x(T_i^-) - \ln H)^2}{4(T_i - t)\sigma^2} - \frac{c^2 t}{2\sigma^2} \right) \right], \\
&= \frac{1}{\sqrt{2\pi\sigma^2}} \sqrt{\pi} \exp \left\{ \frac{c(x(T_i^-) - \ln H)}{\sigma^2} \right\} \frac{\exp \left(-\left| \frac{\sqrt{2}(x(T_{i-1}^+) - \ln H)}{\sigma} \right| \sqrt{s - \frac{c^2}{2\sigma^2}} \right)}{\sqrt{s - \frac{c^2}{2\sigma^2}}}, \\
&= \frac{1}{\sqrt{2\sigma^2}} \exp \left\{ \frac{c(x(T_i^-) - \ln H)}{\sigma^2} \right\} \frac{\exp \left(-\left| \frac{(x(T_{i-1}^+) - \ln H)}{\sigma} \right| \sqrt{2} \sqrt{s - \frac{c^2}{2\sigma^2}} \right)}{\sqrt{s - \frac{c^2}{2\sigma^2}}}.
\end{aligned}$$

Combining results we get:

$$\begin{aligned}
J(s) &= \mathcal{L}(f(t)) \times \mathcal{L}(h(t)) \\
&= \frac{\exp(-rT_{i-1})}{\gamma} \times \exp \left(\frac{c(x(T_{i-1}^+) - \ln H)}{\sigma^2} \right) \exp \left(-\frac{\sqrt{2}(x(T_{i-1}^+) - \ln H)}{\sigma} \right) \\
&\quad \times \sqrt{s + \frac{c^2}{2\sigma^2}} \times \frac{1}{\sqrt{2\sigma^2}} \exp \left(\frac{c(x(T_i^-) - \ln H)}{\sigma^2} \right) \\
&\quad \times \frac{\exp \left(-\left| \frac{(x(T_{i-1}^+) - \ln H)}{\sigma} \right| \sqrt{2} \sqrt{s - \frac{c^2}{2\sigma^2}} \right)}{\sqrt{s - \frac{c^2}{2\sigma^2}}}, \\
&= \frac{\exp(-rT_{i-1})}{\gamma} \times \exp \left(\frac{-c(x(T_{i-1}^+) - x(T_i^-))}{\sigma^2} \right) \exp \left(-\frac{(x(T_{i-1}^+) - \ln H)}{\sigma} \right) \\
&\quad \times \sqrt{2s + \frac{c^2}{\sigma^2}} \times \frac{1}{\sqrt{2\sigma^2}} \frac{\exp \left(-\left| \frac{(x(T_{i-1}^+) - \ln H)}{\sigma} \right| \sqrt{2} \sqrt{s - \frac{c^2}{2\sigma^2}} \right)}{\sqrt{s - \frac{c^2}{2\sigma^2}}}, \\
&= \frac{\exp(-rT_{i-1})}{\gamma} \times \exp \left(\frac{-c(x(T_{i-1}^+) - x(T_i^-))}{\sigma^2} \right) \exp \left(-\frac{(x(T_{i-1}^+) - \ln H) \times c}{\sigma^2} \right) \\
&\quad \times \sqrt{\frac{\sigma^2}{c^2} (2s + \frac{c^2}{\sigma^2})} \times \frac{1}{\sqrt{2\sigma^2}} \frac{\exp \left(-\left| \frac{(x(T_{i-1}^+) - \ln H)}{\sigma} \right| \sqrt{2} \sqrt{s - \frac{c^2}{2\sigma^2}} \right)}{\sqrt{s - \frac{c^2}{2\sigma^2}}}, \\
&= \frac{\exp(-rT_{i-1})}{\gamma} \times \exp \left\{ -\frac{c}{\sigma^2} \left[(x(T_{i-1}^+) - x(T_i^-)) + (x(T_{i-1}^+) - \ln H) \sqrt{\frac{2s\sigma^2}{c^2} + 1} \right] \right\} \times \\
&\quad \frac{1}{\sqrt{2\sigma^2}} \frac{\exp \left(-\left| \frac{(x(T_{i-1}^+) - \ln H)}{\sigma} \right| \sqrt{2} \sqrt{s - \frac{c^2}{2\sigma^2}} \right)}{\sqrt{s - \frac{c^2}{2\sigma^2}}},
\end{aligned}$$

Finally resulting in:

$$J(s) = \exp(-rT_{i-1}) \times \frac{\exp\left(-\frac{c}{\sigma^2}x\left[(T_{i-1}^+) - x(T_{i-1}^-) + [x(T_{i-1}^+) - \ln H]\sqrt{\frac{2\sigma^2 s}{c^2} + 1}\right]\right)}{\sigma\sqrt{2}\gamma} \times \frac{\exp\left(-\left|\frac{x(T_{i-1}^-) - \ln H}{\sigma}\right|\sqrt{2}\sqrt{s + \frac{c^2}{2\sigma^2} - r}\right)}{\sqrt{s + \frac{c^2}{2\sigma^2} - r}}.$$

Wow that was a lot of work, however we are not finished yet! Reverse transforming from [8] we get our approximation:

Case 1 $x(T_i^-) > \ln H$:

$$J(\tau) = \exp(-rT_{i-1}) \exp\left(-\frac{2[x(T_{i-1}^+) - \ln H][x(T_i^-) - \ln H]}{\tau\sigma^2}\right) + \frac{r[x(T_{i-1}^+) - \ln H \exp(rT_i - 2rT_{i-1})]}{8\sigma}(A_1 + C_1B),$$

Case2 $x(T_i^-) \leq \ln H$:

$$J(\tau) = \exp(-rT_{i-1}) + \frac{r[x(T_{i-1}^+) - \ln H \exp(rT_i - 2rT_{i-1})]}{8\sigma}(A_2 + C_2B).$$

where:

$$\begin{aligned} A_1 &= \frac{2r}{\sigma}\tau(x(T_{i-1}^+) - x(T_i^-)) \exp\left(-\frac{2[x(T_{i-1}^+) - \ln H][x(T_i^-) - \ln H]}{\tau\sigma^2}\right), \\ A_2 &= \frac{2r}{\sigma}\tau(x(T_{i-1}^+) - x(T_i^-) - 2\ln H), \\ B &= 8 - 2r\tau + \frac{2r}{\sigma}\tau(x(T_{i-1}^+) - x(T_i^-))(x(T_{i-1}^+) - x(T_i^-) - 2\ln H), \\ C_1 &= \sqrt{2\pi\tau} \exp\left(\frac{(x(T_{i-1}^+) - x(T_i^-))^2}{2\tau\sigma^2}\right) \left[\Theta\left(\frac{x(T_{i-1}^+) + x(T_i^-) - 2\ln H}{\sqrt{2\tau\sigma^2}}\right) - 1\right], \\ C_2 &= \sqrt{2\pi\tau} \exp\left(\frac{(x(T_{i-1}^+) - x(T_i^-))^2}{2\tau\sigma^2}\right) \left[\Theta\left(\frac{x(T_{i-1}^+) + x(T_i^-)}{\sqrt{2\tau\sigma^2}}\right) - 1\right], \end{aligned}$$

such that $\Theta(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^z \exp\left(-\frac{y^2}{2}\right) dy$ this is the cumulative standard normal distribution. I was unable to derive the inverse transformation analytically, so I implemented a numerical integral checker in my code using the trapezoidal rule to approximate integrals, as outlined in [1]. I have calculated the error between the Taylor series approximation and the trapezoidal method for a given amount of Monte Carlo Simulations outlined in Table 5, which shows that it is a fairly accurate approximation Below is the C++ implementation of the trapezium rule.

```
#include "trapezium.h"
```

```
using namespace std;
```

```

long double trapezium_rule(double Time1, double Time2, const std::
    function<double(double)>& f, int n) { //we are inserting the
    gi funciton
    double h = (Time2 - Time1) / n;
    long double sum = 0.0;

    for (int i = 1; i < n; ++i) { // We are excluding the endpoint
        in the intergral approximation as the gi function can not be
        evaluated at these points
        double t = Time1 + i * h;
        sum += f(t);
    }

    return sum * h;
}

```

Monte Carlo Simulations	Average Error between Approximation to Taylor series
100	0.031981
1000	0.0663503
10000	0.0676997

TABLE 5. Average error between Trapezoidal approximation and Taylor series

Below is the Monte Carlo approach implemented in C++, along with the header files for all the classes used.

Standard Monte Carlo approximation with Jumps.

```
double DownAndOut::StandardMonteCarlo(MJD stock){

    std::vector<double> Times;
    Times = stock.JumpTimes(); //generates exponentially distributed
                               jump times
    double StockPrice= stock.GetS0();
    int i = 0;
    bool Checker = 1;
    double TimeStep = 100;

    for (int i = 0; i + 1 < Times.size(); ++i) {
        double TimeIncrement = Times[i + 1] - Times[i];
        double dt = TimeIncrement / TimeStep;

        for (int z = 0; z < TimeStep; ++z) {
            StockPrice = stock.Dynamics(StockPrice, dt);
            double t = Times[i] + z * dt;
            if (StockPrice < H) {
                return Rebate * std::exp(-stock.GetRF() * t);
            }
        }

        if (i + 1 < Times.size() - 1) {

            double Jumpsize = stock.GetJumpDynamics();
            StockPrice *= std::exp(Jumpsize);

            if (StockPrice < H) {
                return Rebate * std::exp(-stock.GetRF() * Times[i +
                    1]);
            }
        }
    }

    return Payoff(StockPrice) * std::exp(-stock.GetRF()) * Rebate;
}
```

Stock Header file.

```

#ifndef STOCK_H
#define STOCK_H

#include <vector>
#include <cmath>

class Stock { //so for the same stock we can price with black
              scholes.
protected:
    double sigma;
    double riskfree;
    double StartPrice;

public:
    Stock(double initprice, double rf_, double sigma_)
        : StartPrice(initprice), riskfree(rf_), sigma(sigma_) {}
    virtual ~Stock() {}

    double GetS0(){return StartPrice;}
    double GetSigma() { return sigma; }
    double GetRF() const { return riskfree; }
};

class JumpSize {
private:
    double ExpectedValueJump;

public:
    double JumpSigma;
    double JumpMu;
    JumpSize(double mu_, double sigma_)
        : JumpSigma(sigma_), JumpMu(mu_) { SetExpectedValueJump(); }

    void SetExpectedValueJump();
    double GetExpectedValueJump() { return ExpectedValueJump; }
    double JumpDynamics();
};

class MJD : public Stock {
private:
    double lambda; //Frequency of jump
    double c;
    double ExpectedValueJump; // Jump size constant
    JumpSize jump;

```

```

public:
    MJD(double initprice, double riskfree, double sigma_,
        double lambda_, double Jumpmu, double JumpSig)
        : Stock(initprice, riskfree, sigma_), lambda(lambda_), jump(
            Jumpmu, JumpSig) { ExpectedValueJump = jump.
            GetExpectedValueJump(); SetC(); }
    std::vector<double> JumpTimes();
    double Dynamics(double Value, double Increment);
    double GetLogS0() { return std::log(StartPrice); }
    double GetJumpDynamics() { return jump.JumpDynamics(); }
    double ContinuousDynamics(double Start, double t1, double t2);
        // Returns the endpoint of the continuous process
    void SetC();
    double GetC() { return c; }
    double GetSigma() { return sigma; }
    double GetRF() const { return riskfree; }
    double GetJumpMu(){return jump.JumpMu;}
    double GetJumpSig(){ return jump.JumpSigma; }
    double GetLamda(){return lambda;}
    std::vector<double> StockPrices(std::vector<double> times);
    void VisulizeStock();
};

```

Option Header file.

```

#ifndef OPTION_H
#define OPTION_H
#include "Stock.h"
#include "EstimateGI.h"

class Option{
protected:
double Strike;
public:
Option(double k_) : Strike(k_) {}
// virtual double Payoff(double FinalVal) =0;

};

class Barrier : public Option {
protected:
double H; //barrier
double Rebate;
public:
Barrier(double H_, double K_, double R_) : Option(K_) , H(H_),
Rebate(R_) {}
double PriceByMJD_Uniform(MJD stock); //Uniform sampling pricing
implementation
double PriceByMJD_Taylor(MJD stock); //Approximating the price
by taylor expansion on gi intergral
virtual long double NoCrossingDensity(MJD stock,double A,double
B, double t1, double t2) =0; //Probability of stock not
crossing in the brownian bridge
virtual double evaluate_gi(MJD stock, double a, double b, double
t, double T1, double T2) =0;
virtual double gamma(MJD stock,double a, double b, double T1,
double T2) =0;
virtual double Payoff(double FinalVal) =0; // what happens to
this then
virtual long double Call_trapezium(MJD stock, double a, double b,
double T1, double T2)=0;
// double PriceByBSM(Stock stock);
std::vector<double>UniformVarRedCall(MJD stock);

};

```

```

class Call : public Option{
private:
    double T = 1;
    double d1;
    double d2;
    Stock stock;
    void Setd1();
    void Setd2();
public:
    Call(double k, Stock s) : Option(k), stock(s) {
        Setd1();
        Setd2();
    }
    double ClosedPrice();
};

class DownAndOut : public Barrier{
public:
    DownAndOut(double H_, double K_, double R_) : Barrier(H_, K_,
        R_) {}
    double Payoff(double FinalVal) override;
    double evaluate_gi(MJD stock, double a, double b, double t,
        double T1, double T2) override;
    double gamma(MJD stock,double a, double b, double T1, double
        T2) override;
    static double CheckGI(MJD stock, double a, double b, double t,
        double T1, double T2);
    long double Call_trapezium(MJD stock, double a, double b,
        double T1, double T2) override;
    long double NoCrossingDensity(MJD stock,double A,double B,
        double t1, double t2) override;
    double StandardMonteCarlo(MJD stock);

};

#endif

```

Randomizer Header file.

```
#ifndef RANDOM_GENERATOR_H
#define RANDOM_GENERATOR_H

#include <random>

class RandomGenerator{
public:
    static std::mt19937& getGenerator(){
        static std::mt19937 gen(std::random_device{}());
        return gen;
    }
};

#endif
```

REFERENCES

- [1] Milton Abramowitz and Irene A. Stegun. *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. Dover Publications, 1972.
- [2] Maciej J. Capinski and Tomasz Zastawniak. *Numerical Methods in Finance with C++*. Mastering Mathematical Finance. Cambridge University Press, 2012.
- [3] Marek Capinski and Ekkehard Kopp. *The Black-Scholes Model*. Mastering Mathematical Finance. Cambridge University Press, 2012.
- [4] Marek Capinski, Ekkehard Kopp, and Janusz Traple. *Stochastic Calculus for Finance*. Mastering Mathematical Finance. Cambridge University Press, 2012.
- [5] I.S. Gradshteyn and I.M. Ryzhik. *Table of Integrals, Series, and Products*. Academic Press, San Diego, 2000.
- [6] Kazuhisa Matsuda. Introduction to merton jump diffusion model. Technical report, The City University of New York, 2004.
- [7] R.C. Merton. Option pricing when the underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1-2):125-144, 1976.
- [8] S.A.K. Metwally and A.F. Atiya. Using brownian bridge for fast simulation of jump-diffusion process. *The Journal of Derivatives*, 2002.
- [9] L.C.G. Rogers and D. Williams. *Diffusions, Markov Processes and Martingales, Volume 1*. Wiley, New York, 1994.
- [10] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2008.